CHƯƠNG 6
KIẾN TRÚC ỨNG DỤNG AI TẠO SINH

Sau khi học xong chương này, người học có thể:

- *Mô tả được kiến trúc tổng thể của một ứng dụng AI tạo sinh theo mô hình phân tầng, bao gồm vai trò và trách nhiệm của từng tầng.*
- *Phân tích được vai trò và đặc điểm của bốn thành phần chính trong hệ thống AI tạo sinh: mô hình (model), API, dữ liệu (data) và pipeline.*
- *So sánh được ba mô hình kiến trúc phổ biến (API-based, Microservices, RAG-based) và lựa chọn được mô hình phù hợp cho một bài toán cụ thể.*
- *Phân tích được các trade-off cốt lõi trong thiết kế kiến trúc AI (chất lượng – độ trễ, chi phí – năng lực, đơn giản – linh hoạt, kiểm soát – tiện lợi) và áp dụng khung ra quyết định kiến trúc 5 bước.*
- *Thiết kế được kiến trúc cho một ứng dụng AI tạo sinh ở mức độ cơ bản, bao gồm việc lựa chọn công nghệ cho từng thành phần và lập luận cho các quyết định thiết kế.*

6.1 Kiến trúc tổng thể của ứng dụng AI tạo sinh

Sau khi hoàn thành mục này, sinh viên có thể:

- *Mô tả được kiến trúc tổng thể của một ứng dụng AI tạo sinh, bao gồm các tầng chức năng và mối quan hệ giữa chúng.*
- *Phân biệt được ứng dụng AI tạo sinh với ứng dụng phần mềm truyền thống về mặt kiến trúc.*
- *Giải thích được vai trò và trách nhiệm của từng tầng trong kiến trúc phân tầng của ứng dụng AI tạo sinh.*
- *Nhận diện được các thách thức kiến trúc đặc thù khi xây dựng ứng dụng có tích hợp LLM.*

6.1.1 Khởi động: Điều gì làm nên một ứng dụng AI tạo sinh?

Hãy so sánh hai ứng dụng: một ứng dụng web truyền thống hiển thị danh sách sản phẩm từ cơ sở dữ liệu, và ChatGPT – một ứng dụng có thể trả lời câu hỏi, viết mã, sáng tác thơ và phân tích tài liệu. Về bề ngoài, cả hai đều là ứng dụng web với giao diện người dùng, API backend và cơ sở dữ liệu. Nhưng bên dưới bề mặt, kiến trúc của chúng khác biệt một cách căn bản.

Trong ứng dụng truyền thống, hành vi của hệ thống được xác định bởi mã nguồn: mỗi đầu vào dẫn đến một đầu ra có thể dự đoán được thông qua các quy tắc nghiệp vụ được lập trình tường minh. Trong ứng dụng AI tạo sinh, một phần đáng kể "logic" của hệ thống nằm trong mô hình ngôn ngữ lớn (LLM) – một thực thể có hành vi mang tính

xác suất, không hoàn toàn tiền định và có khả năng tạo ra các đầu ra đa dạng từ cùng một đầu vào.

Sự khác biệt này không chỉ là vấn đề về công nghệ – nó đòi hỏi một cách tiếp cận kiến trúc khác. Nếu như kiến trúc phần mềm truyền thống xoay quanh việc tổ chức mã nguồn, quản lý trạng thái và đảm bảo tính nhất quán của dữ liệu, thì kiến trúc ứng dụng AI tạo sinh còn phải giải quyết thêm những câu hỏi mới: làm thế nào để quản lý tính không xác định của đầu ra? Làm thế nào để kết hợp sức mạnh của LLM với các ràng buộc nghiệp vụ? Làm thế nào để đảm bảo chất lượng khi mô hình có thể "bịa ra" thông tin?

Chương này sẽ giúp bạn hiểu cách thiết kế kiến trúc cho ứng dụng AI tạo sinh – từ bức tranh tổng thể đến các mô hình kiến trúc cụ thể, từ các thành phần hệ thống đến những đánh đổi cần cân nhắc khi thiết kế.

6.1.2 Sự khác biệt căn bản giữa ứng dụng truyền thống và ứng dụng AI tạo sinh

Trước khi đi vào chi tiết kiến trúc, cần hiểu rõ những khác biệt nền tảng giữa ứng dụng phần mềm truyền thống và ứng dụng AI tạo sinh. Những khác biệt này chi phối mọi quyết định kiến trúc sau này.

a. Tính xác định và tính xác suất

Trong ứng dụng truyền thống, cùng một đầu vào luôn tạo ra cùng một đầu ra (trừ khi có yếu tố ngẫu nhiên được lập trình tường minh). Điều này cho phép kiểm thử chính xác, gỡ lỗi theo vết và đảm bảo chất lượng thông qua các quy trình xác định.

Trong ứng dụng AI tạo sinh, LLM là một hệ thống xác suất: cùng một prompt có thể tạo ra các phản hồi khác nhau qua mỗi lần gọi. Ngay cả với tham số temperature = 0, vẫn có những biến thiên nhỏ do bản chất của quá trình sinh token. Điều này đặt ra thách thức cho việc kiểm thử, giám sát và đảm bảo chất lượng – những hoạt động vốn dựa trên giả định về tính lặp lại của kết quả.

b. Nguồn gốc của "logic" hệ thống

Trong ứng dụng truyền thống, logic nghiệp vụ nằm trong mã nguồn, được viết bởi lập trình viên, được lưu trong repository và được kiểm soát phiên bản. Logic này có thể được đọc, phân tích và sửa đổi một cách chính xác.

Trong ứng dụng AI tạo sinh, một phần logic nằm trong trọng số của mô hình – thứ không thể "đọc" hay "sửa" trực tiếp. Thay vào đó, lập trình viên điều khiển hành vi của hệ thống thông qua prompt engineering, lựa chọn mô hình, cấu hình tham số và thiết kế pipeline. Điều này tạo ra một lớp trừu tượng mới: "logic" không còn là mã nguồn thuần túy mà là sự kết hợp giữa mã nguồn, prompt và hành vi nổi lên (emergent behavior) của mô hình.

c. Chi phí vận hành và độ trễ

Một truy vấn cơ sở dữ liệu trong ứng dụng truyền thống thường hoàn thành trong vài mili-giây. Một lời gọi LLM có thể mất từ vài trăm mili-giây đến hàng chục giây, tùy thuộc vào độ dài đầu ra, kích thước mô hình và tải của dịch vụ. Chi phí cho mỗi lời gọi LLM cũng cao hơn nhiều bậc so với một truy vấn cơ sở dữ liệu.

Những khác biệt này buộc kiến trúc sư phải suy nghĩ khác về caching, xử lý bất

đồng bộ, quản lý tài nguyên và tối ưu hóa chi phí. Một kiến trúc không tính đến những yếu tố này có thể dẫn đến chi phí vận hành vượt khỏi tầm kiểm soát hoặc trải nghiệm người dùng kém do độ trễ quá cao.

6.1.3 Kiến trúc phân tầng của ứng dụng AI tạo sinh

Mặc dù mỗi ứng dụng AI tạo sinh có những đặc thù riêng, hầu hết đều tuân theo một kiến trúc phân tầng (layered architecture) với bốn đến năm tầng chức năng chính. Hình 6.1 (mô tả bằng văn bản) minh họa kiến trúc này.

Tầng Trải nghiệm (Experience Layer) Giao diện người dùng (Web, Mobile, Chat), voice interface, API endpoint
Tầng Điều phối (Orchestration Layer) Quản lý hội thoại, prompt pipeline, chain/tool execution, state management
Tầng AI/Mô hình (AI/Model Layer) LLM serving, model routing, embedding, fine-tuning, guardrails
Tầng Dữ liệu & Tri thức (Data & Knowledge Layer) Vector database, knowledge graph, document store, cache, context store
Tầng Hạ tầng (Infrastructure Layer) Compute (GPU/TPU), networking, monitoring, logging, security

Hình 6.1: Các tầng kiến trúc cơ bản của ứng dụng AI tạo sinh

a. Tầng Trải nghiệm (Experience Layer)

Đây là tầng tiếp xúc trực tiếp với người dùng cuối. Khác với ứng dụng truyền thống nơi giao diện chủ yếu hiển thị dữ liệu có cấu trúc, tầng trải nghiệm của ứng dụng AI tạo sinh phải xử lý:

- **Hiển thị nội dung sinh**: văn bản định dạng (Markdown, code block, bảng biểu), hình ảnh, âm thanh, video – tất cả đều được sinh ra động và có thể thay đổi sau mỗi lần tương tác.
- **Tương tác hội thoại**: duy trì luồng đối thoại nhiều lượt, hiển thị trạng thái "đang sinh" (streaming), cho phép người dùng dừng hoặc chỉnh sửa giữa chừng.
- **Cơ chế phản hồi**: cho phép người dùng đánh giá chất lượng đầu ra (thumbs up/down), báo cáo lỗi hoặc yêu cầu sinh lại – những tín hiệu quan trọng để cải thiện hệ thống.
- **Đa phương thức**: kết hợp đầu vào văn bản, hình ảnh, giọng nói và đầu ra tương ứng.

Tầng này thường được triển khai dưới dạng web application (React, Next.js), mobile app, hoặc tích hợp vào các nền tảng nhắn tin (Slack, Teams, Zalo).

b. Tầng Điều phối (Orchestration Layer)

Đây là "bộ não điều hành" của ứng dụng AI tạo sinh, chịu trách nhiệm điều phối luồng xử lý từ khi nhận yêu cầu đến khi trả về kết quả. Đây cũng là tầng phức tạp nhất và là nơi tập trung phần lớn logic đặc thù của ứng dụng.

Các trách nhiệm chính của tầng điều phối bao gồm:

- **Quản lý hội thoại (Conversation Management)**: duy trì lịch sử hội thoại, quản

lý ngữ cảnh, xử lý giới hạn context window thông qua các chiến lược như cửa sổ trượt (sliding window) hoặc tóm tắt định kỳ (rolling summary).

- **Thực thi pipeline (Pipeline Execution)**: điều phối chuỗi các bước xử lý – từ việc truy xuất ngữ cảnh (context retrieval), xây dựng prompt, gọi LLM, xử lý hậu kỳ đầu ra, đến việc gọi các công cụ hoặc API bên ngoài.
- **Quản lý trạng thái (State Management)**: theo dõi trạng thái của các tác vụ đang thực thi, xử lý lỗi và thử lại, quản lý các tác vụ bất đồng bộ dài hạn.
- **Định tuyến mô hình (Model Routing)**: quyết định sử dụng mô hình nào cho từng loại yêu cầu dựa trên độ phức tạp, chi phí và yêu cầu về độ trễ. Ví dụ: dùng mô hình nhẹ cho câu hỏi đơn giản, mô hình mạnh cho suy luận phức tạp.
- **Kiểm soát chất lượng (Quality Control)**: áp dụng guardrails để lọc nội dung không phù hợp, xác nhận định dạng đầu ra, phát hiện hallucination và kích hoạt cơ chế fallback khi cần.

Các framework phổ biến cho tầng điều phối bao gồm LangChain, LlamaIndex, Semantic Kernel và các giải pháp tự xây dựng (custom orchestration). Việc lựa chọn framework phụ thuộc vào độ phức tạp của ứng dụng và yêu cầu về kiểm soát.

c. Tầng AI/Mô hình (AI/Model Layer)

Đây là tầng chứa các mô hình AI – "động cơ" của toàn bộ hệ thống. Tầng này bao gồm:

- **Model Serving**: cơ sở hạ tầng để phục vụ mô hình trong môi trường production, bao gồm cân bằng tải (load balancing), tự động mở rộng (auto-scaling) và quản lý phiên bản mô hình.
- **Model Gateway / Router**: một lớp trung gian tiếp nhận yêu cầu từ tầng điều phối, định tuyến đến mô hình phù hợp và trả về kết quả. Gateway có thể tích hợp nhiều nhà cung cấp mô hình (OpenAI, Anthropic, Google, model tự host) thông qua một giao diện thống nhất.
- **Embedding Service**: dịch vụ chuyển đổi văn bản thành vector embeddings, phục vụ cho tìm kiếm ngữ nghĩa và RAG (Retrieval-Augmented Generation).
- **Guardrails**: các cơ chế bảo vệ ở mức mô hình, bao gồm lọc nội dung độc hại, phát hiện PII (Personally Identifiable Information), kiểm soát chủ đề và ngăn chặn prompt injection.
- **Model Registry & Versioning**: quản lý các phiên bản mô hình, theo dõi hiệu năng và cho phép rollback khi cần.

Một quyết định kiến trúc quan trọng ở tầng này là lựa chọn giữa việc sử dụng API bên thứ ba (OpenAI API, Anthropic API, Google Vertex AI) hay tự host mô hình (self-hosting với vLLM, TGI, Ollama). Quyết định này ảnh hưởng đến chi phí, độ trễ, khả năng kiểm soát và yêu cầu về bảo mật dữ liệu.

d. Tầng Dữ liệu & Tri thức (Data & Knowledge Layer)

Tầng này quản lý dữ liệu và tri thức mà ứng dụng cần để hoạt động – từ dữ liệu người dùng đến cơ sở tri thức cho RAG. Các thành phần chính bao gồm:

- **Vector Database**: lưu trữ và tìm kiếm vector embeddings cho RAG. Các lựa chọn phổ biến: Pinecone, Weaviate, Milvus, Qdrant, pgvector (PostgreSQL extension), Chroma.

- **Document Store**: lưu trữ tài liệu gốc (PDF, Markdown, HTML) trước khi được chunk và embedding. Có thể là object storage (S3, GCS) hoặc database chuyên dụng.
- **Knowledge Graph**: biểu diễn mối quan hệ giữa các thực thể, hỗ trợ suy luận phức tạp và truy xuất ngữ cảnh có cấu trúc. Đặc biệt hữu ích cho các ứng dụng đòi hỏi độ chính xác cao.
- **Context Store / Cache**: lưu trữ ngữ cảnh hội thoại, kết quả truy vấn thường xuyên và embedding đã tính toán để giảm độ trễ và chi phí.
- **Feature Store**: lưu trữ và phục vụ các đặc trưng (features) cho các mô hình ML bổ trợ, ví dụ: mô hình phân loại ý định, mô hình phát hiện cảm xúc.

e. Tầng Hạ tầng (Infrastructure Layer)

Tầng cuối cùng cung cấp nền tảng vận hành cho toàn bộ hệ thống:

- **Compute**: GPU/TPU cho model serving (nếu self-host), CPU cho các dịch vụ khác. Các nền tảng đám mây như AWS (SageMaker, Bedrock), GCP (Vertex AI), Azure (Azure AI Studio) cung cấp giải pháp quản lý.
- **Networking**: kết nối giữa các dịch vụ, API gateway, load balancer, CDN cho static assets.
- **Monitoring & Observability**: theo dõi độ trễ, throughput, tỉ lệ lỗi, chi phí token, chất lượng đầu ra. Các công cụ như LangSmith, Weights & Biases, Datadog, Grafana.
- **Security**: quản lý API key, xác thực và phân quyền, mã hóa dữ liệu, audit log.
- **CI/CD**: pipeline triển khai cho cả mã nguồn ứng dụng và cấu hình mô hình/prompt.

6.1.4 Nguyên lý thiết kế cốt lõi cho kiến trúc AI tạo sinh

Từ những khác biệt căn bản đã nêu, một số nguyên lý thiết kế quan trọng nổi lên khi xây dựng kiến trúc cho ứng dụng AI tạo sinh:

a. Thiết kế cho tính không xác định

Vì đầu ra của LLM không hoàn toàn xác định, kiến trúc cần được thiết kế để chấp nhận và xử lý sự biến thiên này. Các chiến lược cụ thể bao gồm:

- Sử dụng output parsing có khả năng chịu lỗi (fault-tolerant parsing) thay vì expect định dạng chính xác tuyệt đối.
- Triển khai cơ chế retry với prompt được điều chỉnh khi đầu ra không đạt yêu cầu.
- Áp dụng structured output (JSON mode, function calling) để ràng buộc định dạng đầu ra khi cần tích hợp với các hệ thống khác.
- Thiết kế fallback path cho trường hợp mô hình không thể tạo ra đầu ra hợp lệ sau nhiều lần thử.

b. Tối ưu hóa cho độ trễ và chi phí

Chi phí và độ trễ của lời gọi LLM cao hơn nhiều bậc so với các thao tác phần mềm truyền thống. Các chiến lược tối ưu bao gồm:

- **Caching thông minh**: cache kết quả cho các prompt phổ biến hoặc tương tự về mặt ngữ nghĩa (semantic caching).

- **Streaming**: sử dụng token streaming để cải thiện trải nghiệm người dùng, cho phép hiển thị kết quả từng phần thay vì đợi toàn bộ phản hồi.
- **Mô hình phân tầng**: sử dụng mô hình nhẹ (như GPT-4o mini, Claude Haiku, Gemini Flash) cho các tác vụ đơn giản và chỉ dùng mô hình mạnh cho các tác vụ phức tạp thực sự cần suy luận sâu.
- **Batch processing**: gom nhóm các yêu cầu không cần phản hồi tức thời để xử lý theo batch, tận dụng các mức giá rẻ hơn (như batch API của OpenAI với discount 50%).
- **Xử lý bất đồng bộ**: chuyển các tác vụ nặng (sinh báo cáo dài, phân tích tài liệu lớn) sang xử lý bất đồng bộ với cơ chế thông báo khi hoàn thành.

c. Kiểm soát và giám sát toàn diện

Do tính chất "hộp đen" của LLM, việc giám sát và kiểm soát trở nên đặc biệt quan trọng:

- **Observability toàn diện**: theo dõi không chỉ các chỉ số hệ thống (độ trễ, lỗi) mà còn cả các chỉ số chất lượng AI (độ chính xác, mức độ hallucination, mức độ liên quan).
- **Guardrails đa tầng**: triển khai cơ chế bảo vệ ở nhiều tầng – từ input filtering, prompt inspection, output validation đến post-processing.
- **Audit trail**: ghi lại toàn bộ tương tác với LLM (prompt, response, model version, timestamp) để phục vụ gỡ lỗi, phân tích và tuân thủ.
- **Human-in-the-loop**: thiết kế các điểm kiểm soát nơi con người có thể can thiệp, đặc biệt cho các quyết định có tác động lớn.

d. Module hóa và khả năng thay thế

Công nghệ AI đang phát triển với tốc độ chưa từng có. Mô hình tốt nhất hôm nay có thể bị thay thế trong vài tháng. Kiến trúc cần được thiết kế để:

- Cô lập sự phụ thuộc vào mô hình cụ thể sau các interface trừu tượng.
- Cho phép thay đổi mô hình (model swapping) mà không ảnh hưởng đến phần còn lại của hệ thống.
- Dễ dàng thử nghiệm A/B testing giữa các mô hình hoặc phiên bản prompt khác nhau.
- Hỗ trợ cập nhật prompt và cấu hình mà không cần triển khai lại toàn bộ ứng dụng.

6.1.5 Ví dụ minh họa: Kiến trúc một ứng dụng hỏi đáp tài liệu

Để cụ thể hóa kiến trúc phân tầng, hãy xem xét một ứng dụng RAG (Retrieval-Augmented Generation) điển hình: hệ thống hỏi đáp trên tài liệu nội bộ của doanh nghiệp.

1. **Tầng Trải nghiệm**: người dùng đặt câu hỏi qua giao diện chat web. Câu hỏi được gửi đến backend qua WebSocket để hỗ trợ streaming.
2. **Tầng Điều phối**: khi nhận câu hỏi, orchestrator thực hiện các bước:
   (a) Chuyển câu hỏi thành embedding query thông qua embedding service.
   (b) Truy vấn vector database để tìm các đoạn tài liệu liên quan nhất (top-k retrieval).
   (c) Xây dựng prompt cuối cùng: kết hợp system prompt, các đoạn tài liệu

truy xuất được (context), lịch sử hội thoại và câu hỏi hiện tại.
   (d) Gọi LLM qua model gateway với prompt đã xây dựng.
   (e) Stream kết quả về cho người dùng.
   (f) Lưu tương tác vào context store để dùng cho các lượt hội thoại tiếp theo.
3. **Tầng AI/Mô hình**: embedding service (có thể là OpenAI text-embedding-3-small hoặc model tự host) và LLM service (có thể là GPT-4o, Claude Sonnet hoặc model khác) xử lý yêu cầu. Model gateway định tuyến dựa trên loại yêu cầu.
4. **Tầng Dữ liệu**: vector database (như Pinecone hoặc pgvector) lưu trữ embedding của tài liệu; document store (S3) lưu trữ tài liệu gốc; context store (Redis) cache embedding và kết quả truy vấn thường xuyên.
5. **Tầng Hạ tầng**: toàn bộ hệ thống chạy trên Kubernetes cluster với GPU nodes cho model serving (nếu self-host), được giám sát bởi Datadog và LangSmith.

Qua ví dụ này, bạn có thể thấy mỗi tầng đều có vai trò rõ ràng và sự tương tác giữa các tầng tạo nên một hệ thống hoàn chỉnh. Trong thực tế, ranh giới giữa các tầng không phải lúc nào cũng rạch ròi – ví dụ, một số ứng dụng có thể gộp tầng Điều phối và tầng AI/Mô hình thành một khối, hoặc triển khai vector database như một phần của tầng Hạ tầng. Tuy nhiên, việc phân tách khái niệm thành các tầng như trên giúp bạn suy nghĩ một cách có hệ thống về kiến trúc và dễ dàng trao đổi với các thành viên trong nhóm.

Phần tiếp theo sẽ đi sâu vào từng thành phần hệ thống, phân tích chi tiết về mô hình, API, dữ liệu và pipeline trong kiến trúc ứng dụng AI tạo sinh.

6.2 Các thành phần hệ thống AI tạo sinh

Sau khi hoàn thành mục này, sinh viên có thể:

- *Phân tích được vai trò và đặc điểm của từng thành phần trong hệ thống AI tạo sinh: mô hình, API, dữ liệu và pipeline.*
- *So sánh được các phương án khác nhau cho từng thành phần (model-as-a-service vs self-hosting, các loại database cho AI, các kiểu pipeline).*
- *Giải thích được vòng đời của một yêu cầu trong ứng dụng AI tạo sinh, từ khi người dùng gửi câu hỏi đến khi nhận được phản hồi.*
- *Lựa chọn được công nghệ và cách tiếp cận phù hợp cho từng thành phần dựa trên yêu cầu cụ thể của ứng dụng.*

6.2.1 Khởi động: "Con quái vật" nhiều đầu

Khi bắt đầu xây dựng một ứng dụng AI tạo sinh, nhiều lập trình viên có xu hướng tập trung toàn bộ sự chú ý vào LLM – "bộ não" của hệ thống. Đây là một sai lầm phổ biến. LLM chỉ là một thành phần trong một hệ sinh thái phức tạp hơn nhiều. Nếu không được bao quanh bởi các thành phần hỗ trợ phù hợp, ngay cả mô hình mạnh nhất cũng sẽ tạo ra trải nghiệm kém: phản hồi chậm, thông tin lỗi thời, thiếu ngữ cảnh hoặc thậm chí gây hại.

Hãy hình dung mỗi thành phần trong hệ thống AI tạo sinh như một "đầu" của con rồng – mỗi đầu có một chức năng riêng nhưng phải hoạt động đồng bộ để tạo nên sức mạnh tổng thể. Mục này sẽ "mổ xẻ" từng thành phần chính: mô hình (model), API, dữ

liệu (data) và pipeline – giúp bạn hiểu rõ vai trò, các lựa chọn công nghệ và những cân nhắc khi thiết kế từng thành phần.

6.2.2 Mô hình (Model) – Trái tim của hệ thống

Thành phần mô hình là nơi diễn ra quá trình "suy nghĩ" của ứng dụng. Các quyết định liên quan đến mô hình ảnh hưởng trực tiếp đến chất lượng đầu ra, chi phí vận hành và kiến trúc tổng thể.

a. Phân loại mô hình theo vai trò trong ứng dụng

Trong một ứng dụng AI tạo sinh, không phải lúc nào cũng chỉ có một mô hình duy nhất. Thông thường, nhiều mô hình được sử dụng đồng thời, mỗi mô hình đảm nhận một vai trò khác nhau:

- **Mô hình chính (Primary Model)**: mô hình tạo sinh chính, chịu trách nhiệm tạo ra đầu ra cuối cùng cho người dùng. Đây thường là các LLM mạnh nhất như GPT-4o, Claude Opus, Gemini Pro.
- **Mô hình nhẹ (Lightweight Model)**: mô hình nhỏ hơn, nhanh hơn và rẻ hơn, được sử dụng cho các tác vụ đơn giản như phân loại ý định, tóm tắt ngắn, hoặc tiền xử lý prompt. Ví dụ: GPT-4o mini, Claude Haiku, Gemini Flash.
- **Mô hình chuyên biệt (Specialized Model)**: mô hình được huấn luyện hoặc tinh chỉnh cho một tác vụ cụ thể. Ví dụ: embedding model (text-embedding-3-small, bge-large), reranking model (Cohere Rerank), moderation model (OpenAI Moderation).
- **Mô hình đa phương thức (Multimodal Model)**: mô hình có khả năng xử lý nhiều loại dữ liệu đầu vào/đầu ra khác nhau (văn bản, hình ảnh, âm thanh). Ví dụ: GPT-4o (vision + text), Gemini Pro Vision, Claude 4 (image understanding).

Việc lựa chọn mô hình nào cho vai trò nào là một bài toán tối ưu đa mục tiêu: chất lượng, tốc độ và chi phí. Đây cũng chính là ý tưởng cốt lõi của chiến lược sử dụng mô hình phân tầng đã đề cập trong Mục 5.3.

b. Model-as-a-Service vs Self-Hosting

Một trong những quyết định kiến trúc quan trọng nhất là lựa chọn giữa việc sử dụng dịch vụ mô hình từ nhà cung cấp bên thứ ba hay tự vận hành mô hình trên hạ tầng riêng. Bảng 6.1 so sánh hai cách tiếp cận.

Trong thực tế, nhiều tổ chức áp dụng chiến lược kết hợp (hybrid): sử dụng API cho các tác vụ thông thường và mô hình self-host cho các tác vụ nhạy cảm về dữ liệu hoặc có yêu cầu đặc thù về độ trễ.

c. Model Gateway – Lớp trừu tượng hóa mô hình

Khi ứng dụng sử dụng nhiều mô hình từ nhiều nhà cung cấp khác nhau, việc tích hợp trực tiếp với từng API riêng lẻ sẽ dẫn đến mã nguồn phức tạp, khó bảo trì và khó thay đổi. Model Gateway giải quyết vấn đề này bằng cách cung cấp một giao diện thống nhất (unified interface) cho tất cả các mô hình.

Các trách nhiệm của Model Gateway bao gồm:

- **Chuẩn hóa giao diện**: chuyển đổi giữa định dạng yêu cầu nội bộ của ứng dụng

Bảng 6.1: So sánh Model-as-a-Service và Self-Hosting

Tiêu chí	Model-as-a-Service (API)	Self-Hosting
Chi phí khởi tạo	Thấp – trả theo usage, không cần GPU	Cao – cần GPU, hạ tầng, nhân sự vận hành
Chi phí vận hành	Tuyến tính theo usage – có thể cao ở quy mô lớn	Cố định + biến đổi – hiệu quả hơn ở quy mô lớn nếu tối ưu tốt
Độ trễ	Phụ thuộc vào nhà cung cấp, có thể biến động	Kiểm soát được, có thể tối ưu cho use case cụ thể
Kiểm soát chất lượng	Hạn chế – phụ thuộc vào model version của nhà cung cấp	Toàn quyền – tinh chỉnh, quantization, routing tùy chỉnh
Bảo mật dữ liệu	Dữ liệu rời khỏi tổ chức; cần đánh giá rủi ro	Dữ liệu nằm trong hạ tầng của tổ chức
Khả năng mở rộng	Nhà cung cấp lo – có thể bị rate limit	Tự lo – cần thiết kế auto-scaling
Cập nhật mô hình	Tự động – luôn dùng model mới nhất	Thủ công – cần quy trình cập nhật và kiểm thử
Độ sẵn sàng	Phụ thuộc vào SLA của nhà cung cấp	Tự kiểm soát, nhưng cần đầu tư vào high availability

và API cụ thể của từng nhà cung cấp (OpenAI format, Anthropic format, Google format, v.v.).

- **Định tuyến thông minh (Intelligent Routing)**: quyết định mô hình nào sẽ xử lý một yêu cầu cụ thể dựa trên các tiêu chí như loại tác vụ, độ phức tạp, ngân sách token, yêu cầu về độ trễ.
- **Fallback và Retry**: tự động thử lại với mô hình khác nếu mô hình chính gặp lỗi, quá tải hoặc trả về kết quả không đạt yêu cầu.
- **Giám sát tập trung**: thu thập metrics về usage, độ trễ, chi phí và chất lượng cho tất cả các mô hình từ một điểm duy nhất.
- **Quản lý API key**: tập trung hóa việc quản lý và luân chuyển API key cho các nhà cung cấp khác nhau.

Các giải pháp Model Gateway phổ biến bao gồm: LiteLLM (open-source, hỗ trợ 100+ model providers với OpenAI-compatible interface), Portkey (managed gateway với built-in observability), và các custom gateway xây dựng trên nền tảng API management như Kong hoặc Apigee.

6.2.3 API – Cầu nối giữa các thành phần

API trong ứng dụng AI tạo sinh không chỉ là REST endpoint truyền thống. Đặc thù của tương tác với LLM đặt ra những yêu cầu riêng cho thiết kế API.

a. Các loại API trong hệ thống AI tạo sinh

- **Inference API**: API cốt lõi để gọi mô hình – gửi prompt và nhận phản hồi. Hỗ trợ cả chế độ đồng bộ (gửi yêu cầu, đợi phản hồi hoàn chỉnh) và streaming (nhận từng token khi chúng được sinh ra). OpenAI đã thiết lập định dạng chat completion API như một tiêu chuẩn de facto, được hầu hết các nền tảng và công cụ khác tuân

theo.

- **Embedding API**: chuyển đổi văn bản thành vector embedding. Thường được sử dụng nội bộ trong pipeline RAG và không tiếp xúc trực tiếp với người dùng cuối.
- **Management API**: quản lý các tài nguyên AI như fine-tuning jobs, model deployments, file uploads (cho RAG), assistants/agents configuration.
- **Application API**: API hướng đến người dùng cuối hoặc frontend, đóng gói toàn bộ logic nghiệp vụ và điều phối bên trong. Đây là nơi kiến trúc sư định nghĩa các endpoint như `/chat`, `/search`, `/summarize`, v.v.

b. Thiết kế API cho ứng dụng AI – Những điểm cần lưu ý

So với API truyền thống, API cho ứng dụng AI có một số đặc thù cần được xử lý:

- **Streaming response**: thay vì trả về JSON hoàn chỉnh, API cần hỗ trợ Server-Sent Events (SSE) hoặc WebSocket để stream từng token về client, cải thiện đáng kể trải nghiệm người dùng cho các phản hồi dài. Client cần được thiết kế để xử lý streaming, hiển thị từng phần và cập nhật giao diện liên tục.
- **Long-running requests**: một số tác vụ AI (phân tích tài liệu dài, sinh báo cáo) có thể mất hàng chục giây đến vài phút. API nên hỗ trợ mô hình bất đồng bộ: nhận yêu cầu, trả về task ID, và cung cấp endpoint để kiểm tra trạng thái và lấy kết quả khi hoàn thành.
- **Rate limiting và quota**: do chi phí cao của mỗi lời gọi LLM, cần thiết kế rate limiting chặt chẽ hơn so với API truyền thống – không chỉ theo số lượng request mà còn theo số token sử dụng. Cần có cơ chế thông báo cho người dùng khi sắp đạt giới hạn.
- **Error handling đặc thù**: ngoài các lỗi HTTP thông thường, API AI còn cần xử lý các lỗi đặc thù như: content filter triggered (nội dung bị từ chối vì chính sách an toàn), context length exceeded (vượt quá context window), model overloaded (mô hình quá tải). Mỗi loại lỗi cần có chiến lược xử lý và thông báo riêng.
- **Versioning**: khi prompt hoặc model thay đổi, hành vi của API có thể thay đổi đáng kể ngay cả khi interface không đổi. Cần có chiến lược versioning rõ ràng và cơ chế A/B testing để đánh giá tác động trước khi áp dụng rộng rãi.

c. MCP và A2A – Giao thức cho hệ sinh thái agent

Như đã giới thiệu trong Chương 5, MCP (Model Context Protocol) của Anthropic và A2A (Agent-to-Agent) của Google là hai giao thức quan trọng đang định hình cách các thành phần trong hệ thống AI giao tiếp với nhau. MCP chuẩn hóa cách mô hình kết nối với nguồn dữ liệu và công cụ bên ngoài, trong khi A2A chuẩn hóa giao tiếp giữa các agent. Khi thiết kế API cho ứng dụng AI, việc áp dụng các tiêu chuẩn mở này giúp tránh vendor lock-in và tạo điều kiện cho khả năng tương tác (interoperability) trong tương lai.

6.2.4 Dữ liệu – Nhiên liệu của hệ thống AI

Nếu mô hình là "động cơ", thì dữ liệu chính là "nhiên liệu". Chất lượng và cách tổ chức dữ liệu quyết định phần lớn chất lượng đầu ra của ứng dụng AI, đặc biệt trong các hệ thống RAG.

a. Các loại dữ liệu trong ứng dụng AI tạo sinh

- **Dữ liệu người dùng (User Data)**: hội thoại, tùy chọn, phản hồi, lịch sử tương tác. Dữ liệu này thường được lưu trong cơ sở dữ liệu quan hệ (PostgreSQL, MySQL) hoặc NoSQL (MongoDB, DynamoDB).
- **Dữ liệu tri thức (Knowledge Base)**: tài liệu, văn bản, FAQ, hướng dẫn – những gì hệ thống "biết" để trả lời câu hỏi. Đây là thành phần cốt lõi của RAG và thường được lưu trữ dưới dạng document store (S3, GCS) kết hợp với vector database để tìm kiếm ngữ nghĩa.
- **Dữ liệu embedding (Embedding Data)**: vector biểu diễn của văn bản, được lưu trong vector database để tìm kiếm tương tự (similarity search). Đây là cầu nối giữa ngôn ngữ tự nhiên và không gian toán học.
- **Dữ liệu giám sát (Observability Data)**: log, trace, metrics từ tất cả các thành phần trong hệ thống. Dữ liệu này rất quan trọng cho việc gỡ lỗi, tối ưu hóa và đánh giá chất lượng.
- **Dữ liệu huấn luyện và đánh giá (Training & Evaluation Data)**: tập dữ liệu để fine-tune mô hình, tập eval để đánh giá chất lượng prompt và mô hình.

b. Vector Database – Thành phần cốt lõi của RAG

Vector database là một trong những công nghệ quan trọng nhất trong hệ sinh thái AI tạo sinh hiện đại. Nó cho phép tìm kiếm dựa trên sự tương đồng về ngữ nghĩa (semantic similarity) thay vì khớp từ khóa chính xác.

Nguyên lý hoạt động cơ bản:

1. Văn bản (tài liệu, đoạn hội thoại) được chuyển đổi thành vector embedding thông qua embedding model.
2. Vector được lưu trữ trong vector database cùng với metadata (nguồn, thời gian, loại tài liệu).
3. Khi có truy vấn, câu hỏi cũng được chuyển thành embedding.
4. Vector database tìm kiếm các vector gần nhất (nearest neighbor search) dựa trên các độ đo như cosine similarity hoặc Euclidean distance.
5. Các tài liệu tương ứng với các vector gần nhất được trả về làm ngữ cảnh cho LLM.

Bảng 6.2 so sánh một số vector database phổ biến.

c. Chiến lược indexing và chunking

Hiệu quả của RAG phụ thuộc rất nhiều vào cách tài liệu được chia nhỏ (chunking) và lập chỉ mục (indexing). Một số chiến lược phổ biến:

- **Fixed-size chunking**: chia tài liệu thành các đoạn có kích thước cố định (ví dụ: 512 token) với overlap (ví dụ: 50 token). Đơn giản, dễ triển khai nhưng có thể cắt ngang giữa các ý.
- **Semantic chunking**: chia tài liệu dựa trên ranh giới ngữ nghĩa (hết đoạn văn, hết mục). Chất lượng cao hơn nhưng phức tạp hơn để triển khai.
- **Hierarchical chunking**: tạo nhiều mức chunk (đoạn nhỏ, mục, chương), cho phép truy xuất ở nhiều mức độ chi tiết khác nhau.
- **Agentic chunking**: sử dụng LLM để xác định điểm cắt tối ưu dựa trên nội dung.

Bảng 6.2: So sánh một số vector database phổ biến

Database	Loại	Điểm mạnh	Phù hợp cho
Pinecone	Managed cloud	Setup nhanh, hiệu năng cao, không cần vận hành	Startup, dự án vừa và nhỏ, PoC nhanh
Weaviate	Managed / Self-host	Hỗ trợ hybrid search (vector + keyword), GraphQL API	Ứng dụng cần tìm kiếm kết hợp
Milvus	Self-host (có managed)	Hiệu năng cực cao, scale tốt cho hàng tỉ vector	Ứng dụng quy mô lớn, enterprise
Qdrant	Self-host / Managed	Hiệu năng cao, API dễ dùng, viết bằng Rust	Ứng dụng cần performance và flexibility
pgvector	PostgreSQL extension	Tích hợp với PostgreSQL hiện có, không cần infrastructure mới	Ứng dụng đã dùng PostgreSQL
Chroma	Embedded / Self-host	Nhẹ, dễ dùng, phù hợp cho development	Prototyping, dự án nhỏ, development

Chất lượng cao nhất nhưng chi phí indexing cũng cao nhất.

- **Multi-representation indexing**: lưu trữ nhiều biểu diễn của cùng một tài liệu (bản tóm tắt, bản đầy đủ, các đoạn trích chính), phục vụ cho các loại truy vấn khác nhau.

Lựa chọn chiến lược chunking phụ thuộc vào loại tài liệu, độ dài trung bình của truy vấn và yêu cầu về độ chính xác. Không có một chiến lược "tốt nhất" cho mọi trường hợp – đây là một trong những khía cạnh cần thử nghiệm và điều chỉnh liên tục.

6.2.5 Pipeline – Dây chuyền xử lý thông minh

Pipeline là thành phần kết nối tất cả các thành phần khác lại với nhau, định nghĩa luồng xử lý từ đầu vào đến đầu ra. Đây là nơi logic nghiệp vụ của ứng dụng AI được hiện thực hóa.

a. Các mô hình pipeline cơ bản

- **Pipeline tuyến tính (Linear Pipeline)**: các bước được thực thi tuần tự, đầu ra của bước trước là đầu vào của bước sau. Ví dụ: Input → Preprocessing → Embedding → Vector Search → Prompt Construction → LLM Call → Post-processing → Output. Đây là mô hình đơn giản và phổ biến nhất, phù hợp với hầu hết các use case cơ bản.
- **Pipeline phân nhánh (Branching Pipeline)**: pipeline có điều kiện rẽ nhánh dựa trên kết quả của các bước trung gian. Ví dụ: nếu phân loại ý định xác định đây là câu hỏi về tài liệu nội bộ, pipeline rẽ sang nhánh RAG; nếu là câu hỏi kiến thức chung, gọi trực tiếp LLM; nếu là yêu cầu thực thi hành động, rẽ sang nhánh tool-use.
- **Pipeline vòng lặp (Loop / Agentic Pipeline)**: pipeline có khả năng lặp lại các

bước dựa trên kết quả đánh giá. Ví dụ: LLM sinh câu trả lời → đánh giá chất lượng → nếu chưa đạt, quay lại sinh lại với prompt điều chỉnh. Đây chính là hiện thực của mẫu Phản tư (Reflection) đã đề cập trong Chương 5.

- **Pipeline song song (Parallel Pipeline)**: nhiều nhánh xử lý chạy đồng thời, sau đó kết quả được tổng hợp. Ví dụ: gửi cùng một câu hỏi đến nhiều nguồn tri thức khác nhau (vector DB, knowledge graph, SQL database), sau đó tổng hợp kết quả từ tất cả các nguồn.

b. Các thành phần của một pipeline hiện đại

Một pipeline AI tạo sinh hiện đại thường bao gồm các thành phần sau:

- **Prompt Template**: định nghĩa cấu trúc prompt với các placeholder cho biến động (user query, context, history). Tách biệt prompt khỏi mã nguồn cho phép điều chỉnh prompt mà không cần triển khai lại ứng dụng.
- **Context Retrieval**: truy xuất ngữ cảnh liên quan từ các nguồn dữ liệu (vector DB, knowledge graph, API bên ngoài).
- **Tool / Function Calling**: cho phép LLM gọi các hàm hoặc API bên ngoài khi cần thiết (tra cứu thời gian thực, truy vấn cơ sở dữ liệu, gửi email).
- **Output Parser**: chuyển đổi đầu ra thô của LLM thành định dạng có cấu trúc (JSON, XML, typed objects) để các hệ thống downstream có thể tiêu thụ.
- **Guardrails / Validator**: kiểm tra đầu ra trước khi trả về cho người dùng, đảm bảo nội dung an toàn, phù hợp và đúng định dạng.
- **Memory / State**: duy trì trạng thái và ngữ cảnh qua các lượt tương tác, cho phép hội thoại nhiều lượt có ý nghĩa.

c. Ví dụ: Pipeline RAG chi tiết

Để minh họa cách các thành phần pipeline phối hợp với nhau, Hình 6.2 (mô tả bằng văn bản) trình bày pipeline RAG chi tiết cho một ứng dụng hỏi đáp:

Mỗi bước trong pipeline này có thể được bật/tắt hoặc cấu hình độc lập, cho phép linh hoạt điều chỉnh theo đặc thù của từng ứng dụng. Ví dụ: một ứng dụng FAQ đơn giản có thể bỏ qua bước Query Rewriting và Reranking; một ứng dụng y tế có thể thêm bước xác minh y khoa (medical fact-checking) trước khi trả về kết quả.

6.2.6 Tổng kết: Bức tranh thành phần

Bảng 6.3 tóm tắt bốn thành phần chính của hệ thống AI tạo sinh và các quyết định thiết kế then chốt cho từng thành phần.

Hiểu rõ từng thành phần và mối quan hệ giữa chúng là nền tảng để thiết kế kiến trúc tổng thể. Phần tiếp theo sẽ xem xét các mô hình kiến trúc phổ biến – những "bản thiết kế mẫu" mà bạn có thể tham khảo và điều chỉnh cho ứng dụng của mình.

Pipeline RAG – Luồng xử lý một câu hỏi người dùng	
1. **Input Guard**: kiểm tra câu hỏi đầu vào (phát hiện nội dung độc hại, PII, prompt injection). Nếu vi phạm → từ chối và trả về thông báo phù hợp. 2. **Intent Classification** (Lightweight Model): phân loại ý định – hỏi tài liệu nội bộ? kiến thức chung? yêu cầu hành động? → quyết định nhánh pipeline. 3. **Query Rewriting** (LLM): viết lại câu hỏi để tối ưu cho việc truy xuất (giải quyết đại từ, mở rộng từ viết tắt, thêm ngữ cảnh từ lịch sử). 4. **Embedding**: chuyển câu hỏi đã viết lại thành vector embedding. 5. **Vector Search**: tìm top-k đoạn tài liệu liên quan nhất từ vector database. 6. **Reranking** (Specialized Model): sắp xếp lại các đoạn tài liệu theo mức độ liên quan thực sự với câu hỏi (cross-encoder reranking chính xác hơn vector similarity đơn thuần). 7. **Context Filtering**: lọc bỏ các đoạn có điểm liên quan dưới ngưỡng, tránh đưa thông tin nhiễu vào prompt. 8. **Prompt Assembly**: kết hợp system prompt, các đoạn ngữ cảnh đã chọn, lịch sử hội thoại và câu hỏi thành prompt hoàn chỉnh. 9. **LLM Inference** (Primary Model): gọi LLM với prompt đã xây dựng, stream kết quả. 10. **Output Validation**: kiểm tra đầu ra – có trả lời đúng câu hỏi không? có dựa trên ngữ cảnh được cung cấp không? có hallucination không? 11. **Output Guard**: kiểm tra an toàn đầu ra trước khi gửi về người dùng. 12. **Response + Citations**: trả về câu trả lời kèm trích dẫn nguồn tài liệu (nếu có).	

Hình 6.2: Pipeline RAG chi tiết cho ứng dụng hỏi đáp tài liệu

Bảng 6.3: Tổng kết các thành phần hệ thống AI tạo sinh

Thành phần	**Vai trò chính**	**Quyết định thiết kế then chốt**
Mô hình (Model)	"Động cơ" – tạo ra đầu ra thông minh	Model-as-a-Service vs Self-host; chọn mô hình nào cho vai trò nào; model gateway
API	"Cầu nối" – kết nối các thành phần	REST/SSE/WebSocket; đồng bộ vs bất đồng bộ; rate limiting; versioning
Dữ liệu (Data)	"Nhiên liệu" – cung cấp tri thức và ngữ cảnh	Vector DB nào; chiến lược chunking/indexing; caching; document store
Pipeline	"Dây chuyền" – điều phối luồng xử lý	Pipeline tuyến tính/phân nhánh/vòng lặp; prompt template management; guardrails

6.3 Các mô hình kiến trúc phổ biến

Sau khi hoàn thành mục này, sinh viên có thể:

- *Mô tả được ba mô hình kiến trúc phổ biến cho ứng dụng AI tạo sinh: API-based, Microservices và RAG-based.*
- *Phân tích được ưu điểm, nhược điểm và ngữ cảnh phù hợp của từng mô hình kiến trúc.*
- *So sánh được các mô hình kiến trúc dựa trên các tiêu chí: khả năng mở rộng, chi phí, độ phức tạp và khả năng bảo trì.*
- *Lựa chọn được mô hình kiến trúc phù hợp cho một bài toán cụ thể và giải thích được lý do.*

6.3.1 Khởi động: Không có "viên đạn bạc" trong kiến trúc AI

Một câu hỏi thường gặp khi bắt đầu xây dựng ứng dụng AI tạo sinh là: "Nên dùng kiến trúc gì?". Câu trả lời – như thường lệ trong kỹ nghệ phần mềm – là "tùy vào hoàn cảnh". Một startup xây dựng chatbot đơn giản sẽ có nhu cầu kiến trúc rất khác so với một ngân hàng tích hợp AI vào hệ thống core banking, hay một công ty công nghệ xây dựng nền tảng AI đa tenant phục vụ hàng nghìn khách hàng.

Tin tốt là qua thực tiễn của cộng đồng và doanh nghiệp, một số mô hình kiến trúc (architectural patterns) đã được định hình và chứng minh hiệu quả. Mục này sẽ giới thiệu ba mô hình phổ biến nhất: kiến trúc dựa trên API trực tiếp (API-based), kiến trúc microservices cho AI, và kiến trúc dựa trên RAG (RAG-based). Mỗi mô hình sẽ được trình bày kèm với phân tích ưu/nhược điểm, ngữ cảnh áp dụng và ví dụ minh họa.

6.3.2 Mô hình 1: Kiến trúc API-based (API-Centric Architecture)

a. Mô tả

Đây là mô hình đơn giản nhất và thường là điểm khởi đầu cho hầu hết các ứng dụng AI tạo sinh. Trong mô hình này, ứng dụng (thường là một web service monolithic hoặc một API backend đơn giản) giao tiếp trực tiếp với LLM API của nhà cung cấp bên thứ ba. Toàn bộ "trí thông minh" nằm trong LLM; backend chủ yếu đóng vai trò proxy – nhận yêu cầu từ client, thêm system prompt và ngữ cảnh, gọi LLM API, và trả về kết quả.

Kiến trúc API-based	
Client (Web/Mobile) → API Gateway → Backend Service → LLM API (OpenAI/Anthropic/Google) Backend Service → Database (PostgreSQL/MongoDB – lưu trữ người dùng, lịch sử chat) Backend Service → Cache (Redis – session, rate limiting)	

Hình 6.3: Kiến trúc API-based – mô hình đơn giản nhất

b. Ưu điểm

- **Triển khai nhanh**: có thể xây dựng MVP trong vài ngày. Backend chỉ cần xử lý xác thực, lưu trữ và gọi API. Không cần hạ tầng GPU hay vận hành mô hình.

- **Chi phí khởi tạo thấp**: không cần đầu tư GPU, không cần kỹ sư ML vận hành mô hình. Chi phí biến đổi theo usage – trả tiền theo những gì bạn dùng.
- **Luôn cập nhật mô hình mới nhất**: nhà cung cấp API liên tục nâng cấp mô hình; ứng dụng của bạn tự động hưởng lợi từ những cải tiến này mà không cần can thiệp.
- **Đơn giản về vận hành**: giảm đáng kể gánh nặng DevOps – không phải quản lý GPU cluster, model serving, load balancing cho inference.

c. Nhược điểm

- **Giới hạn kiểm soát**: bạn không thể tinh chỉnh mô hình (fine-tune) theo nhu cầu đặc thù (trừ khi sử dụng dịch vụ fine-tuning của nhà cung cấp). Prompt engineering là công cụ chính để định hình hành vi.
- **Phụ thuộc vào nhà cung cấp (Vendor Lock-in)**: ứng dụng phụ thuộc vào API, pricing model và SLA của một nhà cung cấp cụ thể. Việc chuyển đổi nhà cung cấp đòi hỏi thay đổi mã nguồn và prompt.
- **Chi phí có thể tăng nhanh ở quy mô lớn**: ở một ngưỡng usage nhất định, việc tự host mô hình có thể rẻ hơn đáng kể so với gọi API. Đặc biệt với các ứng dụng có lưu lượng cao và yêu cầu token lớn (xử lý tài liệu dài, sinh nội dung dài).
- **Rủi ro về quyền riêng tư dữ liệu**: dữ liệu rời khỏi hạ tầng của tổ chức. Mặc dù hầu hết nhà cung cấp cam kết không sử dụng dữ liệu API để huấn luyện, nhưng với các ngành có quy định nghiêm ngặt (y tế, tài chính, chính phủ), đây vẫn là một rào cản.

d. Khi nào nên dùng

Mô hình này phù hợp nhất cho:

- MVP, PoC hoặc ứng dụng giai đoạn đầu.
- Ứng dụng có lưu lượng thấp đến trung bình.
- Startup muốn ra mắt sản phẩm nhanh.
- Ứng dụng không yêu cầu tùy chỉnh mô hình sâu.
- Dự án nội bộ, công cụ hỗ trợ nhóm.

6.3.3 Mô hình 2: Kiến trúc Microservices cho AI

a. Mô tả

Khi ứng dụng AI tạo sinh phát triển về độ phức tạp – nhiều tính năng, nhiều nguồn dữ liệu, nhiều loại mô hình – mô hình monolithic API-based nhanh chóng bộc lộ hạn chế. Kiến trúc microservices giải quyết vấn đề này bằng cách phân tách hệ thống thành các dịch vụ độc lập, mỗi dịch vụ đảm nhận một chức năng cụ thể và giao tiếp với nhau thông qua API (thường là REST hoặc gRPC).

b. Ưu điểm

- **Khả năng mở rộng độc lập**: mỗi service có thể được scale riêng dựa trên nhu cầu thực tế. Ví dụ: RAG Service có thể cần nhiều instance hơn Chat Service nếu ứng dụng thực hiện nhiều truy vấn tìm kiếm.
- **Công nghệ phù hợp cho từng service**: Chat Service có thể viết bằng Python (hệ sinh thái AI phong phú), User Service bằng Go (hiệu năng cao), Document Service bằng Python (thư viện xử lý tài liệu). Mỗi service dùng ngôn ngữ và công

Kiến trúc Microservices cho AI	
Client → API Gateway → Router/Orchestrator → Chat Service (quản lý hội thoại, streaming) → RAG Service (embedding, vector search, reranking) → Model Gateway Service (model routing, provider abstraction) → Document Service (upload, parsing, chunking, indexing) → Evaluation Service (chất lượng đầu ra, A/B testing) → User Service (xác thực, phân quyền, preferences) → Analytics Service (usage metrics, cost tracking) Mỗi service có database riêng (Database per Service pattern) Message Queue (Kafka/RabbitMQ) cho giao tiếp bất đồng bộ	

Hình 6.4: Kiến trúc Microservices cho ứng dụng AI tạo sinh

nghệ tối ưu cho nhiệm vụ của nó.

- **Độc lập triển khai**: có thể cập nhật RAG Service với thuật toán chunking mới mà không cần triển khai lại toàn bộ hệ thống, giảm rủi ro và thời gian triển khai.
- **Cô lập lỗi**: nếu Document Service gặp sự cố, Chat Service vẫn có thể hoạt động (có thể với chức năng bị giới hạn), thay vì toàn bộ hệ thống sập.
- **Phân công nhóm rõ ràng**: mỗi nhóm có thể sở hữu một hoặc một vài service, làm việc độc lập và triển khai theo nhịp độ riêng.

c. Nhược điểm

- **Độ phức tạp vận hành cao hơn nhiều**: cần container orchestration (Kubernetes), service discovery, distributed tracing, centralized logging. DevOps overhead tăng đáng kể.
- **Độ trễ liên service**: mỗi lời gọi giữa các service thêm độ trễ. Một pipeline RAG có thể cần gọi 3–4 service khác nhau, làm tăng tổng thời gian phản hồi.
- **Data consistency phức tạp**: dữ liệu được phân tán qua nhiều database; việc duy trì tính nhất quán đòi hỏi các pattern như Saga hoặc Eventual Consistency.
- **Over-engineering cho ứng dụng nhỏ**: với ứng dụng đơn giản, microservices là quá mức cần thiết và làm chậm đáng kể tốc độ phát triển ban đầu.
- **Chi phí hạ tầng cao hơn**: mỗi service cần tài nguyên riêng, dẫn đến chi phí cơ sở hạ tầng cao hơn so với monolithic ở quy mô nhỏ.

d. Khi nào nên dùng

Mô hình này phù hợp nhất cho:

- Ứng dụng có độ phức tạp cao, nhiều tính năng độc lập.
- Tổ chức có nhiều nhóm phát triển làm việc song song.
- Hệ thống cần scale không đồng đều giữa các thành phần.
- Ứng dụng đã vượt qua giai đoạn MVP và cần kiến trúc bền vững cho tăng trưởng dài hạn.
- Tổ chức đã có kinh nghiệm vận hành microservices.

6.3.4 Mô hình 3: Kiến trúc RAG-based (Retrieval-Augmented Generation Architecture)

a. Mô tả

RAG (Retrieval-Augmented Generation) không chỉ là một kỹ thuật – nó đã trở thành một mô hình kiến trúc hoàn chỉnh, đặc biệt cho các ứng dụng cần trả lời dựa trên tri thức cụ thể (tài liệu nội bộ, cơ sở kiến thức, dữ liệu doanh nghiệp). Trong mô hình này, LLM không hoạt động độc lập mà được "tăng cường" bởi một hệ thống truy xuất thông tin (retrieval system) cung cấp ngữ cảnh liên quan cho mỗi truy vấn.

Kiến trúc RAG-based đã được đề cập một phần trong Mục 6.2 khi thảo luận về vector database và pipeline. Ở đây, chúng ta sẽ nhìn nhận nó như một mô hình kiến trúc tổng thể.

b. Các thành phần cốt lõi của kiến trúc RAG

- **Ingestion Pipeline**: pipeline xử lý tài liệu đầu vào – thu thập tài liệu từ nhiều nguồn (upload, API, web scraping, database sync), parsing (PDF, HTML, Markdown), chunking, embedding và indexing vào vector database. Pipeline này thường chạy bất đồng bộ và có thể được kích hoạt theo lịch hoặc theo sự kiện.
- **Retrieval Engine**: thành phần xử lý truy vấn tìm kiếm – nhận câu hỏi người dùng, viết lại truy vấn (query rewriting), tạo embedding, tìm kiếm vector database, reranking kết quả. Retrieval Engine có thể hỗ trợ nhiều chiến lược tìm kiếm: semantic search (dựa trên embedding), keyword search (BM25), hybrid search (kết hợp cả hai).
- **Generation Engine**: thành phần sinh câu trả lời – xây dựng prompt từ kết quả truy xuất và câu hỏi, gọi LLM, xử lý và trả về kết quả kèm trích dẫn nguồn.
- **Feedback Loop**: cơ chế thu thập phản hồi và cải thiện – ghi nhận đánh giá của người dùng, phân tích các truy vấn không được trả lời tốt, cập nhật chiến lược chunking hoặc indexing dựa trên dữ liệu thực tế.

c. Các biến thể của kiến trúc RAG

RAG không phải là một mô hình "một kích cỡ vừa tất cả". Tùy vào yêu cầu cụ thể, có nhiều biến thể:

- **Naive RAG**: biến thể cơ bản nhất – chunk → embed → search → generate. Phù hợp cho PoC và ứng dụng đơn giản với tập tài liệu nhỏ.
- **Advanced RAG**: bổ sung các kỹ thuật nâng cao như query rewriting, multi-hop retrieval (truy xuất nhiều bước), recursive retrieval (truy xuất đệ quy – dùng kết quả bước trước để truy xuất bước sau). Phù hợp cho ứng dụng yêu cầu độ chính xác cao hơn.
- **Modular RAG**: các thành phần (chunking, embedding, retrieval, reranking, generation) được module hóa và có thể hoán đổi độc lập. Cho phép thử nghiệm và tối ưu từng thành phần riêng biệt.
- **Agentic RAG**: tích hợp RAG với AI Agent – agent có thể tự quyết định khi nào cần truy xuất, truy xuất từ đâu (nhiều nguồn), và tự đánh giá xem thông tin truy xuất được đã đủ chưa hay cần truy xuất thêm. Đây là hướng phát triển tiên tiến nhất của RAG.
- **Graph RAG**: kết hợp RAG với knowledge graph – ngoài vector search, hệ thống còn truy xuất từ đồ thị tri thức, cho phép trả lời các câu hỏi đòi hỏi suy luận về mối quan hệ giữa các thực thể (ví dụ: "Ai là người đứng đầu bộ phận mà nhân

viên X trực thuộc?").

d. Ưu điểm

- **Giảm hallucination đáng kể**: bằng cách cung cấp ngữ cảnh cụ thể từ nguồn dữ liệu đáng tin cậy, RAG "neo" phản hồi của LLM vào sự thật, giảm thiểu việc mô hình tự "bịa" thông tin.
- **Tri thức luôn cập nhật**: không cần fine-tune lại mô hình. Khi tài liệu mới được thêm vào, chúng được indexing và có sẵn cho truy vấn ngay lập tức.
- **Minh bạch và giải thích được**: có thể trích dẫn chính xác nguồn của mỗi thông tin trong câu trả lời, tăng độ tin cậy và cho phép người dùng xác minh.
- **Bảo mật dữ liệu**: tri thức của tổ chức không cần đưa vào quá trình huấn luyện mô hình (và do đó không rời khỏi tầm kiểm soát). Chỉ những đoạn liên quan được đưa vào prompt tại thời điểm inference.

e. Nhược điểm

- **Độ phức tạp của pipeline**: một hệ thống RAG chất lượng cao đòi hỏi nhiều bước xử lý (parsing, chunking, embedding, indexing, retrieval, reranking), mỗi bước đều có thể là nguồn gốc của lỗi hoặc suy giảm chất lượng.
- **Chất lượng phụ thuộc vào dữ liệu đầu vào**: "garbage in, garbage out" – nếu tài liệu nguồn chất lượng kém, không đầy đủ hoặc lỗi thời, RAG sẽ tạo ra câu trả lời kém chất lượng dù mọi thành phần kỹ thuật đều hoạt động tốt.
- **Chi phí lưu trữ và tính toán**: embedding và lưu trữ vector cho hàng triệu tài liệu có chi phí đáng kể. Mỗi lần indexing lại toàn bộ (khi thay đổi embedding model hoặc chiến lược chunking) cũng tốn kém.
- **Thách thức về multimodal**: RAG cho văn bản đã khá trưởng thành, nhưng RAG cho hình ảnh, âm thanh, video hoặc tài liệu có cấu trúc phức tạp (bảng biểu, biểu đồ) vẫn còn nhiều thách thức.

f. Khi nào nên dùng

Mô hình RAG-based phù hợp nhất cho:

- Ứng dụng hỏi đáp trên tài liệu nội bộ, cơ sở tri thức.
- Chatbot chăm sóc khách hàng cần trả lời dựa trên chính sách, FAQ, hướng dẫn sử dụng.
- Hệ thống hỗ trợ nghiên cứu, phân tích tài liệu pháp lý, y khoa.
- Ứng dụng cần độ chính xác cao và khả năng trích dẫn nguồn.
- Bất kỳ ứng dụng nào mà LLM cần "biết" về dữ liệu riêng của tổ chức.

6.3.5 Case study: Lựa chọn kiến trúc cho ba tình huống thực tế

Để minh họa cách lựa chọn mô hình kiến trúc, chúng ta sẽ phân tích ba tình huống thực tế với các yêu cầu khác nhau.

a. Tình huống 1: Startup xây dựng AI writing assistant

Yêu cầu: Một startup muốn xây dựng công cụ hỗ trợ viết email, báo cáo và bài blog. Người dùng nhập ý tưởng, AI giúp phát triển thành văn bản hoàn chỉnh với nhiều tông giọng khác nhau (trang trọng, thân thiện, hài hước). Không cần truy xuất tài liệu

nội bộ.

Phân tích:

- Ứng dụng không yêu cầu truy xuất tri thức từ tài liệu riêng – LLM có thể xử lý tốt với kiến thức có sẵn và prompt engineering.
- Startup cần ra mắt nhanh, nguồn lực hạn chế.
- Yêu cầu chính: sinh văn bản chất lượng cao, streaming, quản lý người dùng và lịch sử.

Lựa chọn: **Kiến trúc API-based**. Backend monolithic đơn giản (Next.js + PostgreSQL + Redis), gọi trực tiếp LLM API, tập trung nguồn lực vào trải nghiệm người dùng và prompt engineering. Khi sản phẩm phát triển và có thêm yêu cầu (ví dụ: fine-tune model cho phong cách viết đặc thù), có thể cân nhắc chuyển dần sang microservices.

b. Tình huống 2: Ngân hàng xây dựng trợ lý nội bộ cho nhân viên

Yêu cầu: Một ngân hàng muốn xây dựng trợ lý AI cho nhân viên, có khả năng trả lời câu hỏi về chính sách, quy trình, sản phẩm dựa trên hàng nghìn tài liệu nội bộ (policy docs, quy trình nghiệp vụ, tài liệu đào tạo). Yêu cầu: bảo mật tuyệt đối (dữ liệu không rời khỏi hạ tầng ngân hàng), độ chính xác cao, khả năng trích dẫn nguồn cho mỗi câu trả lời.

Phân tích:

- Yêu cầu cốt lõi là hỏi đáp dựa trên tài liệu nội bộ – đây chính là use case kinh điển của RAG.
- Bảo mật dữ liệu là yêu cầu sống còn – không thể gửi tài liệu nội bộ lên API bên ngoài.
- Cần trích dẫn nguồn chính xác để nhân viên có thể xác minh.
- Hệ thống sẽ phục vụ hàng nghìn nhân viên, yêu cầu availability cao.

Lựa chọn: **Kiến trúc RAG-based + Self-hosting**. Mô hình LLM được self-host (qua vLLM hoặc TGI) trên GPU cluster trong hạ tầng riêng. Vector database (Milvus hoặc Qdrant) cũng được self-host. Kiến trúc microservices được áp dụng cho các thành phần: Document Ingestion Service, Retrieval Service, Generation Service, User Management Service. Không có dữ liệu nào rời khỏi hạ tầng ngân hàng. Audit log đầy đủ cho mọi truy vấn.

c. Tình huống 3: Công ty SaaS xây dựng nền tảng AI đa tenant

Yêu cầu: Một công ty SaaS muốn xây dựng nền tảng cho phép khách hàng doanh nghiệp tạo chatbot AI tùy chỉnh – mỗi khách hàng có bộ tài liệu riêng, cấu hình prompt riêng, và có thể chọn model provider riêng. Hệ thống phải phục vụ đồng thời hàng trăm khách hàng (tenant) với dữ liệu được cô lập tuyệt đối.

Phân tích:

- Đa tenant với data isolation nghiêm ngặt – mỗi tenant có document store, vector namespace và cấu hình riêng.
- Cần hỗ trợ nhiều model provider khác nhau (một số khách hàng muốn dùng OpenAI, một số muốn Azure, một số yêu cầu model self-host riêng).
- Cần scale linh hoạt – một tenant có thể có lưu lượng cao gấp 100 lần tenant khác.
- Cần billing, usage tracking và rate limiting cho từng tenant.

Lựa chọn: **Kiến trúc Microservices + Model Gateway**. Đây là sự kết hợp giữa mô hình microservices và model gateway đã thảo luận. Các thành phần chính: Tenant Management Service, Multi-tenant RAG Service (với namespace isolation trong vector DB), Model Gateway Service (hỗ trợ nhiều provider, routing thông minh), Billing & Usage Service, Admin Dashboard Service. Kubernetes được sử dụng cho orchestration, Kafka cho event-driven communication giữa các service.

6.3.6 Bảng so sánh các mô hình kiến trúc

Bảng 6.4 tổng kết so sánh ba mô hình kiến trúc theo các tiêu chí quan trọng.

Bảng 6.4: So sánh các mô hình kiến trúc cho ứng dụng AI tạo sinh

Tiêu chí	API-based	Microservices	RAG-based
Độ phức tạp triển khai	Thấp	Cao	Trung bình – Cao
Thời gian ra MVPo	1–2 tuần	2–4 tháng	3–8 tuần
Chi phí khởi tạo	Thấp	Cao (hạ tầng + nhân sự)	Trung bình
Chi phí vận hành (scale lớn)	Cao (API cost)	Trung bình – Cao	Trung bình (nếu self-host LLM)
Khả năng mở rộng	Hạn chế (monolith)	Rất tốt	Tốt (cần thiết kế đúng)
Kiểm soát mô hình	Hạn chế (prompt only)	Linh hoạt	Linh hoạt (kết hợp prompt + retrieval)
Giảm hallucination	Không có cơ chế đặc biệt	Tùy thiết kế	Có cơ chế (neo vào nguồn)
Trích dẫn nguồn	Không	Có thể, tùy thiết kế	Có (tính năng cốt lõi)
Bảo mật dữ liệu	Phụ thuộc vào provider API	Tốt (nếu self-host)	Tốt (dữ liệu trong tầm kiểm soát)
Phù hợp nhất cho	MVP, ứng dụng đơn giản, startup	Hệ thống phức tạp, nhiều team, đa tenant	Hỏi đáp tài liệu, chat-bot tri thức, enterprise search

6.3.7 Từ mô hình đến triển khai

Điều quan trọng cần nhấn mạnh là các mô hình kiến trúc này không loại trừ lẫn nhau. Trong thực tế, một hệ thống AI tạo sinh trưởng thành thường là sự kết hợp của nhiều mô hình:

- Một ứng dụng có thể bắt đầu với kiến trúc API-based cho MVP.
- Khi cần thêm tính năng hỏi đáp tài liệu, tích hợp RAG-based như một thành phần bổ sung.
- Khi hệ thống phát triển và có nhiều nhóm làm việc, dần chuyển sang microservices.

Đây là một hành trình tiến hóa kiến trúc (architectural evolution) – không phải là một quyết định "chọn một lần rồi thôi". Nguyên tắc quan trọng là: **bắt đầu đơn giản, phát triển kiến trúc cùng với sự trưởng thành của sản phẩm và đội ngũ**. Đừng xây dựng microservices cho một ứng dụng chưa có người dùng; cũng đừng giữ monolithic

khi hệ thống đã có hàng trăm nghìn người dùng và hàng chục lập trình viên cùng làm việc.

Phần tiếp theo – phần cuối cùng của chương – sẽ đi sâu vào các nguyên tắc thiết kế kiến trúc và phân tích những đánh đổi (trade-off) mà mọi kiến trúc sư AI cần nắm vững.

6.4 Bài thực hành 6

Sau khi hoàn thành mục này, sinh viên có thể:

- *Vận dụng được các nguyên tắc thiết kế kiến trúc đặc thù cho ứng dụng AI tạo sinh.*
- *Phân tích được các đánh đổi (trade-off) cốt lõi trong thiết kế kiến trúc AI: chất lượng – độ trễ, chi phí – năng lực, đơn giản – linh hoạt.*
- *Áp dụng được khung ra quyết định kiến trúc có hệ thống cho các bài toán thiết kế ứng dụng AI.*
- *Thực hành thiết kế kiến trúc cho một bài toán cụ thể và lập luận được cho các quyết định của mình.*

6.4.1 Khởi động: Nghệ thuật của sự đánh đổi

Trong một buổi phỏng vấn kỹ thuật, một ứng viên được hỏi: "Bạn sẽ thiết kế hệ thống chatbot cho 10 triệu người dùng như thế nào?". Ứng viên tự tin trả lời: "Tôi sẽ dùng Kubernetes, triển khai microservices, self-host GPT-4 scale, dùng Milvus cho vector search, Kafka cho event streaming, và Elasticsearch cho full-text search." Nhà tuyển dụng hỏi tiếp: "Còn nếu bạn chỉ có 2 tuần và 1 lập trình viên?".

Câu chuyện này minh họa một sự thật quan trọng: **không có kiến trúc nào là "tốt nhất" một cách tuyệt đối**. Mỗi quyết định kiến trúc là một sự đánh đổi – bạn được cái này nhưng phải hy sinh cái kia. Thiết kế kiến trúc giỏi không phải là chọn giải pháp "tốt nhất về mặt kỹ thuật", mà là chọn giải pháp **tối ưu cho ràng buộc cụ thể** của dự án: thời gian, ngân sách, nhân sự, yêu cầu chất lượng và bối cảnh kinh doanh.

Mục này sẽ trang bị cho bạn một khung tư duy có hệ thống để phân tích và ra quyết định kiến trúc cho ứng dụng AI tạo sinh, cùng với việc thực hành áp dụng khung này vào các tình huống cụ thể.

6.4.2 Các nguyên tắc thiết kế kiến trúc cho ứng dụng AI

Trước khi đi vào phân tích trade-off, hãy thiết lập các nguyên tắc thiết kế nền tảng – những "kim chỉ nam" giúp định hướng mọi quyết định kiến trúc.

a. Nguyên tắc 1: Bắt đầu từ bài toán, không phải từ công nghệ

Sai lầm phổ biến nhất trong thiết kế kiến trúc AI là bắt đầu với câu hỏi "nên dùng công nghệ gì?" thay vì "cần giải quyết vấn đề gì?". Hãy luôn bắt đầu bằng việc trả lời các câu hỏi sau trước khi nghĩ đến công nghệ:

- Người dùng là ai và họ cần gì? (user persona, jobs-to-be-done)
- Yêu cầu chất lượng là gì? (độ chính xác, độ trễ chấp nhận được, mức độ hallucination có thể dung thứ)
- Ràng buộc là gì? (thời gian, ngân sách, nhân sự, quy định pháp lý, bảo mật)

- Quy mô dự kiến là bao nhiêu? (người dùng, tài liệu, lượt truy vấn/ngày)
- Điều gì sẽ thay đổi trong tương lai? (tính năng mới, mô hình mới, quy mô tăng)

Chỉ sau khi trả lời rõ ràng những câu hỏi này, bạn mới nên bắt đầu đánh giá các lựa chọn công nghệ.

b. Nguyên tắc 2: Thiết kế cho sự thay đổi

Trong lĩnh vực AI tạo sinh, thay đổi là hằng số duy nhất. Mô hình mới ra mắt hàng tuần. Giá token giảm liên tục. Các kỹ thuật mới (như RAG, agent) tiến hóa nhanh chóng. Kiến trúc cần được thiết kế để:

- **Dễ dàng hoán đổi mô hình**: sử dụng abstraction layer cho model provider. Tránh hardcode logic phụ thuộc vào một mô hình cụ thể.
- **Dễ dàng cập nhật prompt**: lưu trữ prompt template bên ngoài mã nguồn (database, file cấu hình, hoặc dịch vụ quản lý prompt chuyên dụng). Cho phép A/B testing giữa các phiên bản prompt.
- **Dễ dàng mở rộng**: thiết kế module để có thể thêm tính năng mới (loại tài liệu mới, nguồn dữ liệu mới, loại mô hình mới) mà không cần viết lại toàn bộ hệ thống.
- **Dễ dàng quan sát**: tích hợp observability ngay từ đầu, không phải là "việc sẽ làm sau". Bạn không thể cải thiện những gì bạn không đo lường được.

c. Nguyên tắc 3: Đơn giản hóa cho đến khi không thể đơn giản hơn

Hãy nhớ nguyên lý Occam's Razor trong kiến trúc phần mềm: giải pháp đơn giản nhất thường là giải pháp tốt nhất. Mỗi thành phần bạn thêm vào hệ thống đều đi kèm với chi phí: viết mã, kiểm thử, triển khai, giám sát, gỡ lỗi và bảo trì. Trước khi thêm một thành phần mới vào kiến trúc, hãy tự hỏi:

- Liệu có cách nào đơn giản hơn để đạt được cùng mục tiêu không?
- Lợi ích mà thành phần này mang lại có xứng đáng với chi phí bổ sung không?
- Có thể trì hoãn quyết định này cho đến khi thực sự cần không? (YAGNI – You Ain't Gonna Need It)

Một heuristic hữu ích: nếu bạn không thể giải thích lý do tại sao một thành phần tồn tại trong kiến trúc cho một lập trình viên junior trong vòng 2 phút, có lẽ bạn chưa thực sự cần nó.

d. Nguyên tắc 4: Ưu tiên trải nghiệm người dùng

Trong ứng dụng AI tạo sinh, ranh giới giữa "vấn đề kỹ thuật" và "vấn đề trải nghiệm" rất mờ nhạt. Một quyết định kiến trúc có thể ảnh hưởng trực tiếp đến cảm nhận của người dùng:

- **Streaming không phải là optional**: người dùng không muốn nhìn vào màn hình trắng trong 10 giây. Streaming token là một trong những quyết định kiến trúc có tác động UX lớn nhất.
- **Thời gian đến token đầu tiên (TTFT)**: đây là chỉ số UX quan trọng hơn tổng thời gian phản hồi. Người dùng cảm thấy hệ thống "nhanh" nếu họ thấy token đầu tiên xuất hiện trong vòng dưới 500ms.
- **Xử lý lỗi thân thiện**: thay vì hiển thị lỗi kỹ thuật ("502 Bad Gateway"), hãy thiết kế thông báo lỗi có ý nghĩa với người dùng và đề xuất hành động tiếp theo.

- **Cơ chế undo/retry**: cho phép người dùng yêu cầu sinh lại, chỉnh sửa câu hỏi hoặc quay lại phiên bản trước của phản hồi.

6.4.3 Năm trade-off cốt lõi trong kiến trúc AI

Mọi quyết định kiến trúc cho ứng dụng AI tạo sinh đều xoay quanh một số trade-off cơ bản. Hiểu rõ những trade-off này giúp bạn ra quyết định có ý thức thay vì vô tình hy sinh điều quan trọng.

a. Trade-off 1: Chất lượng – Độ trễ

Đây là trade-off phổ biến nhất và xuất hiện trong hầu hết mọi quyết định.

- **Chọn mô hình**: mô hình mạnh hơn (GPT-4o, Claude Opus) cho chất lượng cao hơn nhưng độ trễ lớn hơn. Mô hình nhẹ hơn (GPT-4o mini, Claude Haiku) nhanh hơn nhưng có thể thiếu chính xác cho các tác vụ phức tạp.
- **Số lượng bước pipeline**: mỗi bước bổ sung trong pipeline (query rewriting, reranking, validation) cải thiện chất lượng nhưng thêm độ trễ. Pipeline 12 bước có thể cho chất lượng tuyệt vời nhưng thời gian phản hồi 5–10 giây có thể khiến người dùng rời bỏ.
- **Retrieval depth**: truy xuất nhiều tài liệu hơn (top-20 thay vì top-5) tăng khả năng tìm thấy thông tin liên quan nhưng tăng thời gian xử lý và số token trong prompt, dẫn đến độ trễ cao hơn.
- **Streaming vs Batch**: streaming cải thiện UX (người dùng thấy kết quả sớm hơn) nhưng tổng thời gian hoàn thành có thể lâu hơn so với xử lý batch. Streaming cũng phức tạp hơn để triển khai monitoring và error handling.

Chiến lược cân bằng:

- Sử dụng mô hình phân tầng: mô hình nhẹ cho tác vụ đơn giản, mô hình mạnh cho tác vụ phức tạp.
- Thiết kế pipeline với "early exit": nếu kết quả ở bước đơn giản đã đủ tốt, không cần chạy các bước phức tạp tiếp theo.
- Streaming với chunking thông minh: hiển thị phần đầu của câu trả lời ngay khi có thể, trong khi các phần sau vẫn đang được sinh.

b. Trade-off 2: Chi phí – Năng lực

Mô hình mạnh hơn cho kết quả tốt hơn nhưng chi phí cao hơn – đôi khi gấp 10–50 lần so với mô hình nhẹ. Bảng 6.5 minh họa mức chênh lệch chi phí giữa các mô hình (giá tham khảo tháng 7/2025).

Chiến lược cân bằng:

- **Model cascading**: bắt đầu với mô hình rẻ nhất có thể giải quyết tác vụ. Chỉ chuyển lên mô hình đắt hơn nếu mô hình rẻ không đáp ứng được yêu cầu chất lượng.
- **Caching thông minh**: cache kết quả cho các prompt phổ biến hoặc tương tự ngữ nghĩa. Semantic cache có thể tiết kiệm 30–50% chi phí API cho các ứng dụng có nhiều truy vấn tương tự.
- **Batch processing**: sử dụng batch API (với discount 50%) cho các tác vụ không yêu cầu phản hồi tức thời.
- **Self-hosting tại điểm hòa vốn**: tính toán điểm hòa vốn giữa API và self-hosting.

Bảng 6.5: Chi phí token tham khảo cho một số mô hình phổ biến (USD trên 1M token)

Mô hình	Input ($)	Output ($)	Ghi chú
GPT-4o mini	0,15	0,60	Mô hình nhẹ, nhanh, rẻ
GPT-4o	2,50	10,00	Mô hình đa năng cân bằng
Claude Haiku	0,25	1,25	Mô hình nhẹ của Anthropic
Claude Sonnet	3,00	15,00	Cân bằng giữa Haiku và Opus
Claude Opus	15,00	75,00	Mô hình mạnh nhất, suy luận sâu
Gemini Flash	0,15	0,60	Mô hình nhẹ của Google
Gemini Pro	1,25	5,00	Mô hình cân bằng của Google

Với một mức usage nhất định (thường là hàng trăm triệu token/tháng), self-hosting trở nên kinh tế hơn.

c. Trade-off 3: Đơn giản – Linh hoạt

Mô hình kiến trúc càng linh hoạt thì càng phức tạp:

- **Monolith vs Microservices**: monolithic đơn giản để phát triển và triển khai, nhưng khó mở rộng và bảo trì khi hệ thống phát triển. Microservices linh hoạt hơn nhưng phức tạp hơn nhiều để vận hành.
- **Framework vs Custom**: sử dụng framework như LangChain giúp phát triển nhanh hơn, nhưng có thể gây khó khăn khi cần tùy chỉnh sâu. Custom implementation linh hoạt tuyệt đối nhưng tốn thời gian phát triển hơn.
- **Managed Service vs Self-hosted**: managed service (như Pinecone serverless, OpenAI API) đơn giản trong vận hành nhưng hạn chế về kiểm soát. Self-hosted linh hoạt hoàn toàn nhưng đòi hỏi năng lực vận hành.
- **Cứng nhắc vs Động**: pipeline cứng nhắc (các bước cố định) dễ hiểu, dễ gỡ lỗi nhưng không thích ứng được với các tình huống đa dạng. Agentic pipeline (agent tự quyết định bước tiếp theo) linh hoạt nhưng khó kiểm soát và dự đoán.

Chiến lược cân bằng:

- Bắt đầu với lựa chọn đơn giản hơn. Chỉ tăng độ phức tạp khi có bằng chứng rõ ràng rằng sự đơn giản hiện tại đang cản trở mục tiêu.
- Áp dụng "strangler fig pattern": dần thay thế các phần của hệ thống cũ bằng kiến trúc mới thay vì viết lại toàn bộ.
- Giữ các abstraction ở mức tối thiểu cần thiết. Mỗi lớp trừu tượng đều có chi phí về hiểu biết và gỡ lỗi.

d. Trade-off 4: Kiểm soát – Tiện lợi

Trade-off này đặc biệt quan trọng khi lựa chọn giữa việc sử dụng dịch vụ bên ngoài hay tự xây dựng:

- **API bên thứ ba**: tiện lợi – không cần quản lý hạ tầng, luôn có model mới nhất.

Nhưng mất kiểm soát về: model version (nhà cung cấp có thể thay đổi model mà không báo trước), data privacy, rate limit, pricing.

- **Self-hosting**: kiểm soát tuyệt đối – chọn phiên bản mô hình cụ thể, tinh chỉnh theo nhu cầu, dữ liệu không rời khỏi tổ chức. Nhưng mất tiện lợi: cần đội ngũ vận hành, đầu tư GPU, tự xử lý scaling và failover.
- **Managed Vector DB vs Self-hosted Vector DB**: Pinecone serverless không cần quản lý nhưng giới hạn về customization. Milvus self-hosted linh hoạt nhưng cần người vận hành.

Chiến lược cân bằng:

- Đối với dữ liệu không nhạy cảm và yêu cầu không quá đặc thù: dùng managed service để tập trung nguồn lực vào sản phẩm cốt lõi.
- Đối với dữ liệu nhạy cảm hoặc yêu cầu đặc thù cao: self-host các thành phần liên quan đến dữ liệu và mô hình.
- Sử dụng kiến trúc lai (hybrid): API cho các tác vụ thông thường, self-host cho các tác vụ nhạy cảm.

e. Trade-off 5: Tối ưu cục bộ – Tối ưu toàn cục

Một cạm bẫy phổ biến là tối ưu hóa từng thành phần riêng lẻ mà quên mất bức tranh tổng thể:

- Tối ưu retrieval engine để đạt recall 99% có thể làm tăng độ trễ pipeline lên gấp đôi, khiến trải nghiệm người dùng tệ hơn mặc dù mỗi thành phần đều "tốt".
- Tiết kiệm chi phí bằng cách dùng mô hình rẻ cho mọi tác vụ có thể giảm 50% chi phí API, nhưng nếu tỉ lệ lỗi tăng lên và người dùng rời bỏ, tổn thất lớn hơn nhiều.
- Mua GPU mạnh nhất để self-host có thể cho latency thấp nhất, nhưng nếu GPU chỉ được sử dụng 20% thời gian, chi phí trên mỗi request thực tế cao hơn gọi API.

Chiến lược cân bằng:

- Luôn đánh giá tác động của quyết định kiến trúc dựa trên chỉ số end-to-end (thời gian phản hồi tổng thể, tỉ lệ hài lòng của người dùng, tổng chi phí vận hành) chứ không chỉ các chỉ số thành phần riêng lẻ.
- Thiết lập SLO (Service Level Objectives) ở mức hệ thống và để các quyết định tối ưu thành phần phục vụ SLO đó.

6.4.4 Khung ra quyết định kiến trúc

Để ra quyết định kiến trúc một cách có hệ thống, bạn có thể áp dụng khung 5 bước sau:

1. **Xác định yêu cầu và ràng buộc**: liệt kê tất cả yêu cầu chức năng, phi chức năng và ràng buộc của dự án. Phân loại thành "bắt buộc" (must-have) và "mong muốn" (nice-to-have).
2. **Liệt kê các phương án**: với mỗi quyết định kiến trúc (chọn mô hình, chọn mô hình kiến trúc tổng thể, chọn công nghệ), liệt kê ít nhất 2–3 phương án khả dĩ. Đừng chỉ xem xét phương án "phổ biến" hoặc "quen thuộc".
3. **Phân tích trade-off cho từng phương án**: với mỗi phương án, phân tích tác động lên các chiều: chất lượng, độ trễ, chi phí, độ phức tạp, khả năng mở rộng, bảo mật, khả năng bảo trì. Sử dụng ma trận quyết định nếu cần.

4. **Chọn phương án và ghi lại lý do**: **luôn ghi lại lý do** cho mỗi quyết định kiến trúc quan trọng (Architecture Decision Record – ADR). Điều này cực kỳ quan trọng để các thành viên mới hiểu được ngữ cảnh và để sau này có thể đánh giá lại quyết định khi hoàn cảnh thay đổi.
5. **Xác nhận và điều chỉnh**: triển khai phương án đã chọn, đo lường thực tế so với dự đoán, và sẵn sàng điều chỉnh nếu thực tế khác với giả định ban đầu.

6.4.5 Thực hành: Thiết kế kiến trúc cho hệ thống đề xuất khóa học

Để áp dụng những gì đã học, chúng ta sẽ cùng thực hành thiết kế kiến trúc cho một bài toán cụ thể.

a. Bài toán

Một trường đại học muốn xây dựng hệ thống "Cố vấn học tập AI" giúp sinh viên:

- Nhận đề xuất khóa học phù hợp dựa trên ngành học, sở thích, kết quả học tập và yêu cầu tốt nghiệp.
- Hỏi đáp về nội dung, yêu cầu và đánh giá của từng khóa học (dựa trên đề cương và phản hồi của sinh viên khóa trước).
- Lập kế hoạch học tập cho từng học kỳ, đảm bảo đáp ứng điều kiện tiên quyết và yêu cầu tốt nghiệp.

Ràng buộc:

- Dữ liệu khóa học và sinh viên là thông tin nhạy cảm, phải tuân thủ quy định bảo vệ dữ liệu cá nhân.
- Ngân sách hạn chế – trường không có GPU cluster riêng.
- Hệ thống cần sẵn sàng trước kỳ đăng ký môn học tiếp theo (3 tháng).
- Đội ngũ phát triển: 2 backend developer, 1 frontend developer, không có kỹ sư ML chuyên trách.

b. Hướng dẫn phân tích

Bước 1 – Xác định yêu cầu và ràng buộc:

- *Must-have*: bảo mật dữ liệu cá nhân; trả lời chính xác dựa trên dữ liệu khóa học thực tế; trích dẫn nguồn thông tin; thời gian phản hồi dưới 5 giây.
- *Nice-to-have*: cá nhân hóa sâu; tích hợp với hệ thống đăng ký môn học; hỗ trợ giọng nói.
- *Ràng buộc*: không có GPU; không có kỹ sư ML; ngân sách hạn chế; 3 tháng; dữ liệu nhạy cảm.

Bước 2 – Phương án kiến trúc:

- *Phương án A (API-based)*: Backend đơn giản gọi LLM API, lưu dữ liệu khóa học dưới dạng JSON trong prompt. Đơn giản nhất nhưng giới hạn về lượng dữ liệu có thể đưa vào context window.
- *Phương án B (RAG-based + API)*: Sử dụng RAG với vector database để truy xuất thông tin khóa học liên quan, gọi LLM API. Phức tạp hơn nhưng xử lý được lượng dữ liệu lớn và nhiều khóa học.
- *Phương án C (RAG + Self-host)*: Tự host cả LLM và vector DB. Kiểm soát tốt nhất nhưng vượt quá ngân sách và năng lực đội ngũ.

Bước 3 – Phân tích trade-off:

- Phương án C bị loại ngay do không có GPU và kỹ sư ML – vi phạm ràng buộc.
- Phương án A không xử lý được lượng dữ liệu khóa học (có thể lên đến hàng trăm khóa, mỗi khóa có đề cương 5–20 trang) trong context window.
- Phương án B phù hợp nhất: dùng RAG với managed vector DB (Pinecone hoặc pgvector) và LLM API. Dữ liệu nhạy cảm: cần kiểm tra điều khoản bảo mật của nhà cung cấp API. Hầu hết các provider lớn (OpenAI, Anthropic) cam kết không dùng dữ liệu API để huấn luyện và cung cấp tùy chọn zero data retention cho khách hàng doanh nghiệp.

Bước 4 – Quyết định:

- **Kiến trúc tổng thể**: API-based monolithic + RAG (lai giữa hai mô hình).
- **LLM**: Claude Sonnet qua API (cân bằng tốt giữa chất lượng và chi phí) cho câu hỏi phức tạp; GPT-4o mini cho câu hỏi đơn giản và phân loại ý định.
- **Vector DB**: pgvector (tận dụng PostgreSQL hiện có của trường, không cần infrastructure mới).
- **Backend**: Python (FastAPI) – đơn giản, hệ sinh thái AI phong phú.
- **Frontend**: Next.js với React – streaming support tốt.
- **Triển khai**: Docker Compose trên cloud VM (đơn giản, đủ dùng cho quy mô một trường).

Bước 5 – Kế hoạch xác nhận:

- Tuần 1–2: Xây dựng prototype RAG cơ bản với 10 khóa học, kiểm tra chất lượng trả lời.
- Tuần 3–4: Đánh giá độ trễ và chi phí với tải dự kiến (100 sinh viên đồng thời).
- Tuần 5–8: Mở rộng lên toàn bộ khóa học, tích hợp frontend.
- Tuần 9–12: Kiểm thử với sinh viên thực, thu thập phản hồi, điều chỉnh prompt và chunking strategy.

6.4.6 ADR Template – Ghi lại quyết định kiến trúc

Để duy trì tính minh bạch và khả năng truy vết của các quyết định kiến trúc, mỗi quyết định quan trọng nên được ghi lại dưới dạng ADR (Architecture Decision Record). Dưới đây là một template đơn giản:

Bảng 6.6: Template Architecture Decision Record (ADR)

Tiêu đề	Mô tả ngắn gọn về quyết định (ví dụ: "Sử dụng pgvector thay vì Pinecone cho vector search")
Trạng thái	Đề xuất / Đã chấp nhận / Đã triển khai / Đã thay thế
Ngày	Ngày ra quyết định
Bối cảnh	Mô tả vấn đề cần giải quyết, các ràng buộc và yêu cầu liên quan.
Các phương án	Liệt kê các phương án đã xem xét cùng với ưu/nhược điểm của từng phương án.
Quyết định	Phương án được chọn và lý do chi tiết.
Hệ quả	Những tác động của quyết định này: tích cực, tiêu cực, rủi ro, và những gì cần làm tiếp theo.

Việc duy trì ADR không chỉ giúp onboarding thành viên mới mà còn là công cụ

quý giá để đánh giá lại quyết định khi hoàn cảnh thay đổi. Rất nhiều quyết định kiến trúc "sai lầm" thực ra là quyết định đúng đắn tại thời điểm nó được đưa ra, nhưng hoàn cảnh đã thay đổi. ADR giúp bạn hiểu được ngữ cảnh gốc và điều chỉnh phù hợp.

6.4.7 Tổng kết: Kiến trúc là một quá trình, không phải một sự kiện

Thiết kế kiến trúc cho ứng dụng AI tạo sinh không phải là việc bạn làm một lần ở đầu dự án rồi quên đi. Đó là một quá trình liên tục: đánh giá, quyết định, triển khai, đo lường và điều chỉnh. Công nghệ AI thay đổi nhanh chóng có nghĩa là những giả định hôm nay có thể không còn đúng vào ngày mai – và kiến trúc của bạn cần đủ linh hoạt để thích ứng.

Những điểm chính cần ghi nhớ từ chương này:

- Kiến trúc ứng dụng AI tạo sinh khác biệt căn bản với kiến trúc phần mềm truyền thống do tính không xác định của LLM, chi phí và độ trễ cao của inference, và sự phụ thuộc vào mô hình như một nguồn "logic".
- Kiến trúc phân tầng (Experience, Orchestration, AI/Model, Data/Knowledge, Infrastructure) cung cấp một khung tư duy có hệ thống.
- Ba mô hình kiến trúc phổ biến – API-based, Microservices, RAG-based – phục vụ các ngữ cảnh khác nhau và có thể kết hợp.
- Năm trade-off cốt lõi (Chất lượng – Độ trễ, Chi phí – Năng lực, Đơn giản – Linh hoạt, Kiểm soát – Tiện lợi, Tối ưu cục bộ – Toàn cục) là "kim chỉ nam" cho mọi quyết định.
- Khung ra quyết định 5 bước và ADR giúp bạn thiết kế kiến trúc một cách có hệ thống và có thể truy vết.

Với nền tảng kiến trúc vững chắc từ chương này, chương tiếp theo sẽ đi sâu vào thiết kế hệ thống và dữ liệu – hai khía cạnh cụ thể và chi tiết hơn trong việc xây dựng ứng dụng AI tạo sinh.

6.4.8 Thiết kế kiến trúc hệ thống

Xây dựng hệ thống Quản lý bán hàng (tiếp tục từ Bài 5).
Xem chi tiết tại Bài thực hành 1. Nhắc lại: mọi kết quả AI phải được kiểm tra; không cung cấp dữ liệu nhạy cảm cho AI công cộng; duy trì Nhật ký sử dụng AI.

Sinh viên sử dụng AI để đề xuất các phương án kiến trúc phần mềm phù hợp với yêu cầu dự án. Sinh viên lựa chọn kiến trúc, thiết kế các thành phần, mô hình triển khai, cơ chế giao tiếp và cấu trúc dữ liệu; đồng thời phân tích ưu điểm, hạn chế và các đánh đổi của phương án đã chọn.

a. Mục tiêu

- Chuyển yêu cầu chức năng và thuộc tính chất lượng thành quyết định kiến trúc.
- So sánh ít nhất hai phương án bằng tiêu chí cụ thể.
- Mô tả thành phần, trách nhiệm, giao tiếp, dữ liệu và triển khai.
- Ghi nhận quyết định kiến trúc và các đánh đổi.

b. Chuẩn bị

- Bộ yêu cầu cập nhật, quy mô người dùng và các ràng buộc triển khai.
- Thông tin dự kiến về tải, dữ liệu, bảo mật, ngân sách và năng lực nhóm.
- Công cụ vẽ sơ đồ và mẫu ghi quyết định kiến trúc.
- Phiên bản hệ thống hiện tại để đánh giá khả năng chuyển đổi.

c. Hướng dẫn thực hiện

Bước 1: Xác định động lực kiến trúc

- Chọn các Use Case có ảnh hưởng lớn và các thuộc tính chất lượng quan trọng.
- Viết kịch bản chất lượng có nguồn kích thích, sự kiện, môi trường, thành phần chịu tác động và kết quả đo được.
- Ghi rõ ràng buộc về công nghệ, nhân lực, thời gian và chi phí.

Bước 2: Đề xuất phương án

- Yêu cầu AI đề xuất tối thiểu hai phương án phù hợp quy mô dự án.
- Không mặc định kiến trúc phân tán là tốt hơn; xem xét độ phức tạp vận hành.
- Mô tả cách mỗi phương án xử lý dữ liệu, giao tiếp, xác thực, lỗi và mở rộng.

Bước 3: Đánh giá và lựa chọn

- Xây dựng ma trận tiêu chí có trọng số: khả năng bảo trì, hiệu năng, bảo mật, mở rộng, chi phí và năng lực nhóm.
- Chấm điểm dựa trên bằng chứng hoặc giả định được ghi rõ.
- Thực hiện nguyên mẫu kỹ thuật nhỏ nếu có rủi ro chưa chắc chắn.

Bước 4: Mô tả kiến trúc được chọn

- Vẽ sơ đồ bối cảnh, container/khối triển khai và thành phần chính.
- Mỗi thành phần có trách nhiệm, giao diện, dữ liệu sở hữu và phụ thuộc rõ ràng.
- Mô tả luồng cho ít nhất ba kịch bản: tạo đơn hàng, cập nhật tồn kho và lập báo cáo.

Bước 5: Thiết kế vận hành và bảo mật

- Xác định cấu hình, quản lý bí mật, ghi log, giám sát, sao lưu và khôi phục.
- Mô tả ranh giới tin cậy, xác thực, phân quyền và bảo vệ dữ liệu.
- Xác định điểm lỗi và phương án suy giảm an toàn.

Bước 6: Ghi quyết định và kế hoạch chuyển đổi

- Lập bản ghi quyết định kiến trúc gồm bối cảnh, lựa chọn, phương án thay thế và hệ quả.
- Xây dựng các bước chuyển từ hệ thống hiện tại sang kiến trúc mục tiêu.
- Cập nhật sơ đồ khi hiện thực khác với thiết kế.

d. Prompt mẫu

Prompt 1 – So sánh kiến trúc:

"Bạn là kiến trúc sư phần mềm. Dựa trên yêu cầu [nội dung], quy mô [số liệu], ràng buộc [nội dung] và năng lực nhóm [mô tả], hãy đề xuất 2–3 phương án kiến trúc. So sánh theo hiệu năng, bảo mật, bảo trì, mở rộng, độ phức tạp vận hành, chi phí và rủi ro. Nêu giả định và không chọn phương án thay cho nhóm."

Prompt 2 – Phản biện kiến trúc:

"Hãy phản biện kiến trúc sau: [mô tả]. Dùng các kịch bản [danh sách] để tìm điểm nghẽn, phụ thuộc vòng, điểm lỗi đơn, rò rỉ dữ liệu, trách nhiệm chồng chéo và khó khăn triển khai. Xếp hạng rủi ro và đề xuất phép thử để xác minh."

e. Kiểm tra và hiệu chỉnh kết quả AI

- Kiểm tra đề xuất phù hợp quy mô thật, tránh kiến trúc quá phức tạp.
- Xác minh tên kỹ thuật, giới hạn và cơ chế giao tiếp bằng tài liệu chính thức của công nghệ được chọn.
- Không để AI tự đặt số liệu tải hoặc mục tiêu chất lượng.
- Đảm bảo sơ đồ và mã nguồn hiện tại không mâu thuẫn mà không có kế hoạch chuyển đổi.
- Ghi rõ quyết định thuộc về nhóm và lý do không chọn phương án khác.

f. Sản phẩm cần nộp

- Danh sách động lực và kịch bản chất lượng.
- Ma trận so sánh ít nhất hai phương án.
- Bộ sơ đồ kiến trúc và mô tả luồng chính.
- Tối thiểu 3 bản ghi quyết định kiến trúc.
- Kế hoạch chuyển đổi, danh sách rủi ro và Nhật ký sử dụng AI của Bài 6.

g. Tiêu chí đánh giá (10 điểm)

- Động lực và kịch bản chất lượng: 1,5 điểm.
- Phương án và ma trận đánh giá: 2,0 điểm.
- Lựa chọn có căn cứ và phân tích đánh đổi: 1,5 điểm.
- Sơ đồ, trách nhiệm và luồng hệ thống: 2,0 điểm.
- Thiết kế bảo mật, vận hành và xử lý lỗi: 1,5 điểm.
- Quyết định, kế hoạch chuyển đổi và Nhật ký AI: 1,5 điểm.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **Kiến trúc phân tầng của ứng dụng AI tạo sinh:** Năm tầng chức năng – Trải nghiệm (giao diện người dùng, streaming, đa phương thức), Điều phối (quản lý hội thoại, pipeline execution, model routing, quality control), AI/Mô hình (model serving, gateway, embedding, guardrails), Dữ liệu & Tri thức (vector database, document store, knowledge graph, cache), Hạ tầng (compute, networking, monitoring, security). Mỗi tầng có vai trò và trách nhiệm rõ ràng.
- **Sự khác biệt căn bản với ứng dụng truyền thống:** Tính không xác định của đầu ra LLM; "logic" hệ thống nằm một phần trong trọng số mô hình thay vì hoàn toàn trong mã nguồn; chi phí và độ trễ inference cao hơn nhiều bậc so với truy vấn cơ sở dữ liệu. Những khác biệt này chi phối mọi quyết định kiến trúc.
- **Bốn nguyên tắc thiết kế cốt lõi:** (1) Thiết kế cho tính không xác định (fault-tolerant parsing, retry, structured output, fallback); (2) Tối ưu cho độ trễ và chi phí (caching, streaming, mô hình phân tầng, batch processing); (3) Kiểm soát và giám sát toàn diện (observability, guardrails, audit trail, human-in-the-loop); (4) Module hóa và khả năng thay thế (cô lập phụ thuộc mô hình, model swapping, A/B testing).
- **Bốn thành phần hệ thống chính:** Mô hình (Model-as-a-Service vs Self-Hosting, model gateway, phân loại mô hình theo vai trò), API (inference API, streaming/SSE, long-running requests, rate limiting theo token, MCP/A2A), Dữ liệu (vector database, chiến lược chunking/indexing, semantic caching), Pipeline (tuyến tính, phân nhánh, vòng lặp/agentic, song song – với các thành phần: prompt template, context retrieval, tool calling, output parser, guardrails).
- **Ba mô hình kiến trúc phổ biến:** (1) API-based – đơn giản, triển khai nhanh, phù hợp MVP và startup; (2) Microservices – scale độc lập, phân công nhóm rõ ràng, phù hợp hệ thống phức tạp và tổ chức lớn; (3) RAG-based – giảm hallucination, trích dẫn nguồn, tri thức luôn cập nhật, phù hợp ứng dụng hỏi đáp tài liệu. Ba mô hình không loại trừ lẫn nhau và có thể kết hợp.
- **Năm trade-off cốt lõi:** Chất lượng – Độ trễ (mô hình mạnh hơn = chậm hơn); Chi phí – Năng lực (mô hình mạnh có thể đắt gấp 10–50 lần); Đơn giản – Linh hoạt (monolith đơn giản, microservices linh hoạt); Kiểm soát – Tiện lợi (API tiện lợi, self-host kiểm soát); Tối ưu cục bộ – Tối ưu toàn cục (tối ưu từng thành phần có thể làm giảm chất lượng tổng thể).
- **Khung ra quyết định kiến trúc 5 bước:** (1) Xác định yêu cầu và ràng buộc; (2) Liệt kê các phương án; (3) Phân tích trade-off; (4) Chọn phương án và ghi lại lý do (ADR); (5) Xác nhận và điều chỉnh. Kiến trúc là một quá trình liên tục, không phải một sự kiện một lần.

Câu hỏi ôn tập

1. Liệt kê năm tầng trong kiến trúc phân tầng của ứng dụng AI tạo sinh. Với mỗi tầng, nêu một trách nhiệm chính và một ví dụ về công nghệ hoặc thành phần cụ thể.
2. Phân biệt ba điểm khác biệt căn bản giữa ứng dụng phần mềm truyền thống và ứng dụng AI tạo sinh về mặt kiến trúc. Tại sao những khác biệt này lại quan

trọng khi thiết kế kiến trúc?

3. Trình bày chiến lược sử dụng mô hình phân tầng (tiered model) trong kiến trúc AI. Khi nào nên dùng mô hình nhẹ và khi nào cần mô hình mạnh? Cho ví dụ cụ thể về model routing.
4. Model Gateway là gì và nó giải quyết những vấn đề gì trong kiến trúc ứng dụng AI? Nêu ít nhất ba trách nhiệm của Model Gateway.
5. So sánh Model-as-a-Service và Self-Hosting trên ba tiêu chí: chi phí, bảo mật dữ liệu và khả năng kiểm soát. Trong trường hợp nào self-hosting là lựa chọn phù hợp hơn?
6. Mô tả pipeline RAG chi tiết gồm ít nhất 8 bước. Với mỗi bước, giải thích mục đích và một rủi ro tiềm ẩn nếu bước đó hoạt động không tốt.
7. So sánh ba mô hình kiến trúc API-based, Microservices và RAG-based. Với mỗi mô hình, nêu một tình huống cụ thể mà mô hình đó là lựa chọn phù hợp nhất và giải thích lý do.
8. Phân tích trade-off "Chất lượng – Độ trễ" trong kiến trúc ứng dụng AI. Nêu ít nhất hai chiến lược cụ thể để cân bằng trade-off này.
9. Tại sao streaming không nên được coi là một tính năng "optional" trong ứng dụng AI tạo sinh? Phân tích ảnh hưởng của streaming đến trải nghiệm người dùng và nêu những thách thức kỹ thuật khi triển khai streaming.
10. ADR (Architecture Decision Record) là gì và tại sao nó quan trọng trong phát triển ứng dụng AI? Nêu các thành phần chính của một ADR.

Bài tập thực hành

1. **Thiết kế kiến trúc cho hệ thống chatbot chăm sóc khách hàng:** Một công ty thương mại điện tử muốn xây dựng chatbot AI để trả lời câu hỏi của khách hàng về: trạng thái đơn hàng, chính sách đổi trả, thông tin sản phẩm và xử lý khiếu nại. Hệ thống cần tích hợp với database đơn hàng hiện có và kho tài liệu chính sách (khoảng 200 tài liệu). Dự kiến 5.000 lượt hỏi/ngày.
   (a) Vẽ sơ đồ kiến trúc tổng thể cho hệ thống này (có thể dùng text hoặc công cụ vẽ), thể hiện rõ các tầng và thành phần.
   (b) Với mỗi thành phần, đề xuất công nghệ cụ thể và giải thích lý do lựa chọn.
   (c) Phân tích ít nhất ba trade-off trong thiết kế của bạn và cách bạn giải quyết chúng.
   (d) Viết một ADR cho quyết định lựa chọn vector database trong hệ thống này.
2. **Phân tích và cải tiến kiến trúc:** Cho mô tả kiến trúc sau: "Hệ thống hỏi đáp tài liệu nội bộ: Frontend React → Backend Node.js (Express) → OpenAI API (GPT-4o). Tài liệu được lưu trong thư mục Google Drive, mỗi lần có câu hỏi, backend liệt kê tất cả file, tải về, trích xuất văn bản và nhét toàn bộ vào prompt cùng với câu hỏi."
   (a) Chỉ ra ít nhất bốn vấn đề trong kiến trúc này (về hiệu năng, chi phí, chất lượng, khả năng mở rộng).
   (b) Thiết kế lại kiến trúc để giải quyết các vấn đề trên. Mô tả các thay đổi chính và lý do.
   (c) Ước tính mức cải thiện về độ trễ và chi phí giữa kiến trúc cũ và mới cho

một truy vấn điển hình.

3. **Thực hành xây dựng pipeline RAG đơn giản:** Sử dụng Python và các thư viện phù hợp:
   (a) Chọn một bộ tài liệu nhỏ (3–5 file PDF hoặc text) làm cơ sở tri thức.
   (b) Triển khai pipeline RAG cơ bản: chunking, embedding (dùng OpenAI hoặc model miễn phí như all-MiniLM-L6-v2), lưu trữ vector (dùng Chroma hoặc FAISS), truy xuất và sinh câu trả lời.
   (c) Đánh giá chất lượng pipeline với 5 câu hỏi kiểm thử. Ghi nhận: câu trả lời có chính xác không? Có dựa trên tài liệu nguồn không? Có hallucination không?
   (d) Đề xuất ít nhất hai cải tiến cho pipeline của bạn (ví dụ: thêm reranking, cải thiện chunking, query rewriting) và giải thích lý do.
4. **Phân tích chi phí cho các phương án kiến trúc:** Một ứng dụng AI xử lý trung bình 10.000 truy vấn/ngày, mỗi truy vấn cần khoảng 2.000 token input và 500 token output.
   (a) Tính chi phí hàng tháng nếu sử dụng: (a) GPT-4o mini, (b) GPT-4o, (c) Claude Sonnet, (d) Claude Opus. Sử dụng bảng giá tham khảo trong Mục 6.4.
   (b) Tính chi phí self-hosting với một GPU A100 (khoảng $6.000/tháng nếu thuê cloud), giả sử GPU xử lý được 50 truy vấn đồng thời và hoạt động 24/7. So sánh với các phương án API.
   (c) Đề xuất chiến lược model routing (kết hợp nhiều mô hình) để tối ưu chi phí mà vẫn đảm bảo chất lượng. Ước tính chi phí với chiến lược của bạn.

Câu hỏi thảo luận

1. Một số kiến trúc sư cho rằng "kiến trúc microservices cho AI là over-engineering trong hầu hết trường hợp". Dựa trên nội dung chương này và kinh nghiệm của bạn, bạn đồng ý hay phản đối? Lập luận của bạn cần đề cập đến ít nhất hai trade-off đã được trình bày trong chương.
2. Trong tương lai, khi context window của LLM ngày càng lớn (1M, 2M, thậm chí 10M token), liệu RAG có còn cần thiết không? Hay chúng ta có thể chỉ cần "nhét tất cả tài liệu vào prompt"? Phân tích ưu và nhược điểm của cả hai cách tiếp cận, xét trên các khía cạnh: chất lượng, chi phí, độ trễ và khả năng mở rộng.
3. Các giao thức như MCP (Model Context Protocol) và A2A (Agent-to-Agent) đang được đề xuất như những tiêu chuẩn cho hệ sinh thái AI. Bạn có cho rằng những giao thức này sẽ trở thành tiêu chuẩn phổ biến như HTTP/REST trong phát triển web không? Những yếu tố nào sẽ quyết định sự thành công hay thất bại của chúng?